TRAKYA ÜNİVERSİTESİ · BİLGİSAYAR MÜHENDİSLİĞİ

# Dosya Organizasyonu

Hafta 5 · Dosya Sistemleri ve Zincirleme

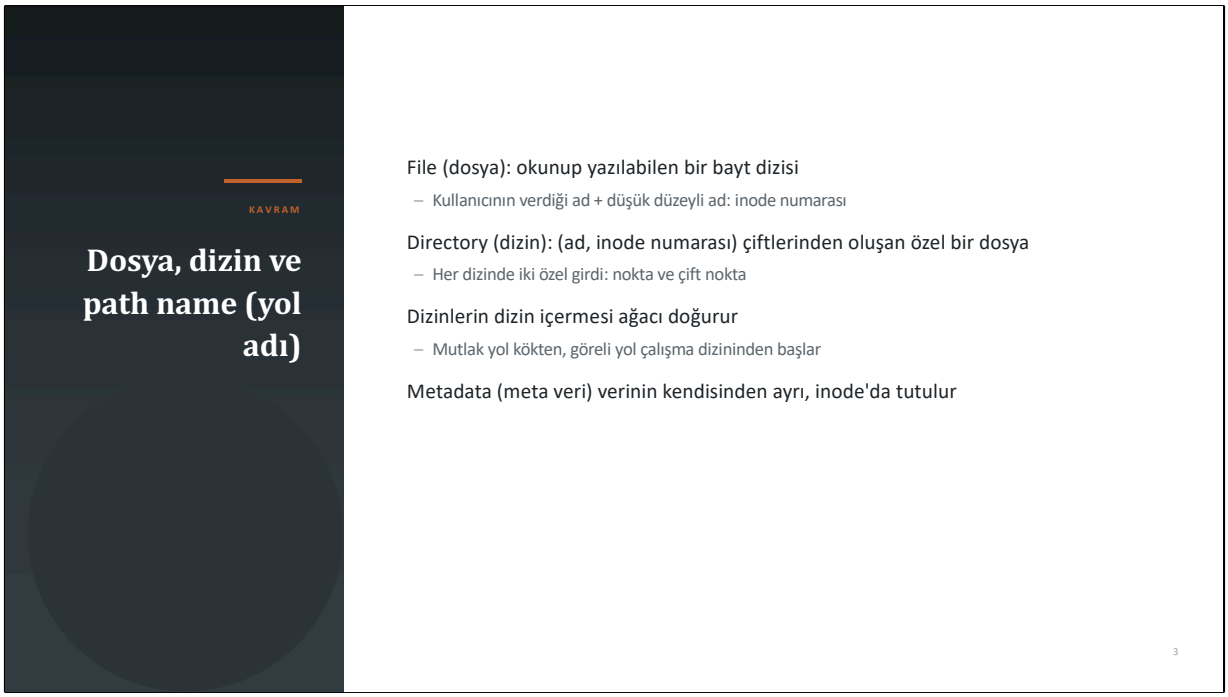
**Doç. Dr. Emir Öztürk**

2025/2026 Güz Yarıyılı

# 1

# Dosya, dizin ve disk düzeni

Soyutlama, bağlantılar ve diskteki bölgeler




File (dosya), üzerinde okuma ve yazma yapılabilen bir bayt dizisidir. Kullanıcının verdiği adın yanında her dosyanın düşük düzeyli bir adı daha bulunur: inode numarası. Directory (dizin) de bir dosyadır, ancak içeriği belirlidir: ad ile inode numarası çiftlerinden oluşan bir listedir ve her dizinde kendisini gösteren nokta ile üst dizini gösteren çift nokta girdileri yer alır. Dizinlerin başka dizinleri içerebilmesi ağaç yapısını doğurur; bir dosyaya kökten ya da çalışma dizininden başlayan bir path name (yol adı) ile ulaşılır. Dosyanın boyu, sahibi, izinleri ve zaman damgaları verinin kendisinden ayrı olarak inode'da saklanır.

BAĞLANTI

# Sabit ve symbolic link (sembolik bağlantı)




Hard link (sabit bağlantı): aynı inode'a ikinci bir ad

- inode'daki sayaç sıfıra inince veri serbest bırakılır: silme adı kaldırır
- Dizinlere ve başka file system (dosya sistemine) kurulamaz

Symbolic link (sembolik bağlantı): içeriği bir yol olan ayrı bir dosya

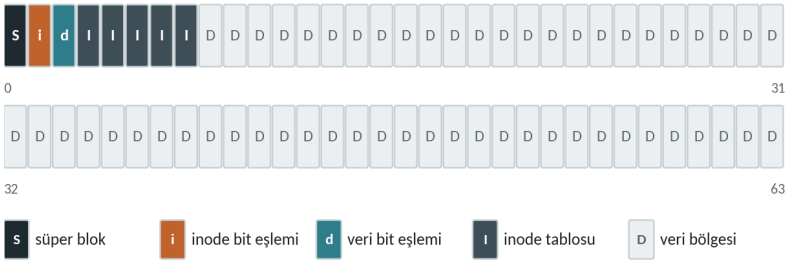
- Hedef silinirse bağlantı sarkar, yol çözülemez

4

Hard link (sabit bağlantı), var olan bir inode numarasına ikinci bir ad bağlamaktır. Dosya kopyalanmaz; bir dizine aynı inode numarasını gösteren yeni bir girdi eklenir ve iki ad arasında asıl ve kopya ayrımı kalmaz. inode içindeki bağlantı sayacı her yeni adla artar, her silmede azalır; veri ve inode ancak sayaç sıfıra indiğinde serbest bırakılır. inode numaraları yalnızca kendi file system (dosya sistemi) içinde tekil olduğundan sabit bağlantı başka bir dosya sistemine kurulamaz. Symbolic link (sembolik bağlantı) bu sınırı aşar: içeriği hedefin yolu olan ayrı bir dosyadır ve hedef silindiğinde sarkan bağlantıya dönüşür.

ÖRNEK

## Basit bir file system (dosya sisteminin) düzeni


S i d I I I I I D D D

HESAP

## Bölgelerin boyutları

| Bölge | Blok | İçerik | Sınır |
|---|---|---|---|
| Süper blok | 1 | dosya sistemi bilgisi | : |
| inode bit eşlemi | 1 | hangi inode boş | : |
| Veri bit eşlemi | 1 | hangi blok boş | : |
| inode tablosu | 5 | 80 inode (256 bayt) | en çok 80 dosya |
| Veri bölgesi | 56 | kullanıcı verisi | 224 KB |

4 KB bloğa 16 inode, beş blok 80 inode: en çok 80 dosya

Superblock (süper blok): inode ve blok sayıları, tablonun başı, tür imzası

– Bağlama sırasında ilk okunan yapıdır

6

Bölgelerin boyutları file system (dosya sisteminin) sınırlarını belirler. Her inode 256 bayt olduğunda 4 kilobaytlık bir bloğa 16 inode sığar; beş blok ayrıldığında toplam 80 inode elde edilir ve bu sayı dosya sisteminde bulunabilecek en çok dosya sayısıdır. Daha büyük bir diskte inode tablosuna daha çok blok ayrılarak sınır yükseltilir. Superblock (süper blok) dosya sisteminin kendisine ilişkin bilgiyi taşır: kaç inode ve kaç veri bloğu bulunduğu, inode tablosunun nerede başladığı ve dosya sisteminin türünü tanıtan imza. Bağlama sırasında ilk okunan yapı olduğundan kimi dosya sistemleri kopyalarını da saklar.

2

# inode

Meta veri ve verinin yerini gösteren işaretçiler

YAPI

## inode numarasından diskteki yere

```
blok    = (inode_no * inode_boyu) / blok_boyu;
sektor  = ((blok * blok_boyu) + inode_baslangic) / sektor_boyu;
```

inode numarası dosyanın düşük düzeyli adıdır

Yapı bir dizidir: ad zaten bir indekstir

Örnek: 32 numaralı inode, 256 baytlık inode, 12 KB'da başlayan tablo

- 32 × 256 = 8192 → 12 KB + 8 KB = 20 KB
- 20 KB ÷ 512 bayt = 40. sektör

Arama tek bir hesaba indirgenmiştir



Her inode bir numarayla anılır. Bu numara dosyanın düşük düzeyli adıdır. Numara verildiğinde inode'un diskteki yeri hesapla bulunur: numara ile inode boyunun çarpımı inode bölgesi içindeki uzaklığı verir, bu uzaklığa bölgenin başlangıç adresi eklenir ve sonuç sektör boyuna bölünerek okunacak sektör elde edilir. Yapının dizi biçiminde olması, aramayı tek bir hesaba indirgemektedir. Adın kendisi zaten bir indekstir.

YAPI

# inode alanları ve direct pointer (doğrudan işaretçi)

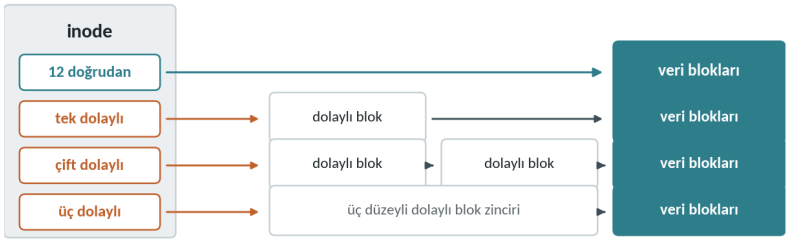
- inode: tür, boy, izinler, sahip, zaman damgaları, bağlantı sayısı
  - Alanların çoğu birkaç bayt; en büyük yeri 15 disk işaretçisi kaplar
- Direct pointer (doğrudan işaretçi): her işaretçi bir veri bloğunu gösterir
  - Küçük dosyada hiçbir ek erişim gerekmez
- Sınır: blok boyu × işaretçi sayısı
  - 12 işaretçi · 4 KB blok → 48 KB
- İşaretçi sayısını artırmak inode'u büyütür: başka yapı gerekir



inode, bir dosyanın meta verisini taşıyan sabit uzunluklu bir kayıttır: dosyanın türü, boyu, kendisine ayrılan blok sayısı, izinleri, sahibi, zaman damgaları ve bağlantı sayısı burada bulunur. Alanların çoğu birkaç bayttır, en büyük yeri disk işaretçileri kaplar. En yalın düzende her işaretçi bir veri bloğunu doğrudan gösterir ve küçük dosyalarda hiçbir ek erişim gerekmez. Dosyanın boyu blok boyu ile işaretçi sayısının çarpımını aşamaz; on iki işaretçi ve 4 kilobaytlık blok için bu sınır 48 kilobayttır. İşaretçi sayısını artırmak her inode'u büyüteceğinden büyük dosyalar için başka bir yapı gereklidir.

TASARIM

# Multi-level index (çok düzeyli indeks)




4 KB blok · 4 baytlık işaretçi → $(12 + 1024 + 1024^2) \times 4\ \text{KB} \approx 4\ \text{GB}$

Indirect pointer (dolaylı işaretçi), veri yerine işaretçi taşıyan bloğu gösterir



Multi-level index (çok düzeyli indeks) yaklaşımında inode'a indirect pointer (dolaylı işaretçiler) eklenir. Dolaylı işaretçi, kullanıcı verisi yerine başka işaretçileri taşıyan bir bloğu gösterir. Tek dolaylı blok, 4 kilobaytlık blok ve 4 baytlık adres varsayımıyla 1024 işaretçi daha kazandırır. Çift dolaylı işaretçi, dolaylı blokları gösteren bir blok ekleyerek sınırı bir kat daha yükseltir. Üç dolaylı işaretçiyle yapı bir düzey daha derinleşir. Ortaya çıkan yapı dengesiz bir ağaçtır.

HESAP

## Düzeylerin kattığı kapasite

| Düzey | İşaretçi sayısı | Kattığı kapasite | Ek okuma |
|---|---|---|---|
| 12 doğrudan | 12 | 48 KB | yok |
| Tek dolaylı | 1 024 | 4 MB | 1 blok |
| Çift dolaylı | $1\,024^2$ | 4 GB | 2 blok |
| Üç dolaylı | $1\,024^3$ | 4 TB | 3 blok |

Varsayım: 4 KB blok, 4 baytlık disk adresi

Derinlik arttıkça veriden önce okunan blok sayısı artar

Dosyaların çoğu küçük: ağaç bilerek dengesiz kurulur

- Küçük dosya yalnızca doğrudan işaretçilerle karşılanır



Tabloda her düzeyin kattığı kapasite ayrı ayrı görülmektedir. Direct pointer (doğrudan işaretçiler) küçük dosyaları ek erişim olmadan karşılar. Tek dolaylı blok sınırı birkaç megabayta, çift dolaylı blok birkaç gigabayta taşır. Derinlik arttıkça veriye ulaşmadan önce dolaylı blokların okunması gerektiğinden erişim maliyeti de artar. Yapının dengesiz olması bilinçli bir seçimdir: file system (dosya sistemleri) üzerindeki ölçümler dosyaların çoğunun birkaç kilobayt olduğunu, buna karşılık yerin çoğunu birkaç büyük dosyanın kapladığını göstermektedir. Sık görülen küçük dosya hiç ek okuma gerektirmez, ender görülen büyük dosya derinleşen yapının maliyetini öder.

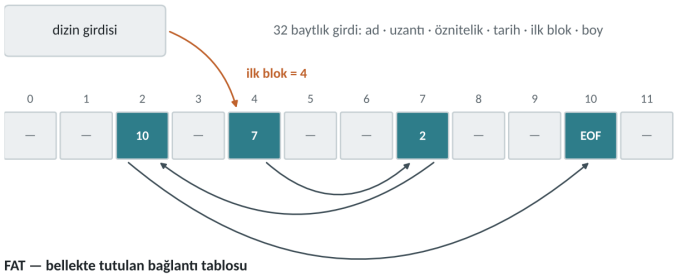
3

# FAT

Bağlı liste yaklaşımının gerçek bir sistemdeki karşılığı

YAPI

# FAT zinciri




FAT — bellekte tutulan bağlantı tablosu

Bağlantı bilgisi veri bloklarından ayrılıp tabloya taşınmıştır

Zincir diskte değil bellekte izlenir: rastgele erişim olanaklı



Dosyanın blokları, bellekte tutulan bir tabloda izlenir. Dizin girdisindeki ilk blok numarası bu tabloya indeks olarak kullanılır. Tablodaki değer bir sonraki bloğun numarasıdır ve zincir böylece sürer. Özel bir değer dosyanın sonunu bildirir. Bağlantı bilgisinin veri bloklarından ayrılıp tabloya taşınması, rastgele erişimi olanaklı kılar. Zincir diskte değil bellekte izlenir. Dosyanın beşinci bloğuna ulaşmak için zincirin ilk dört halkası izlenmek zorundadır. Ancak tablo bellekte bulunduğundan bu izleme disk erişimi gerektirmez. Yalnızca hedef bloğun kendisi diskten okunur. Tablonun diskteki kopyası, güç kesilmesine karşı iki nüsha olarak tutulur.

SÜRÜM

## Sürümler ve tasarımın sonuçları

| Sürüm | Adres | Blok boyu | En büyük bölüm |
|---|---|---|---|
| FAT-12 | 12 bit | 0,5 ile 4 KB | 16 MB |
| FAT-16 | 16 bit | 2 ile 32 KB | 2 GB |
| FAT-32 | 28 bit | 4 ile 32 KB | 2 TB |

Büyük blok geniş bölüm demek, ama internal fragmentation (iç parçalanma) artar

- 32 KB blokta 10 KB'lık dosya yerin üçte ikisini harcar

inode yok: meta veri 32 baytlık dizin girdisinde

- Aynı dosyaya ikinci ad bağlanamaz: hard link (sabit bağlantı) tanımsız

Tablo bellekte tutulur, boyu bölüm boyuyla büyür

14

FAT'in üç sürümü disk adresinin kaç bit olduğuna göre ayrılır; adres genişledikçe desteklenen bölüm boyu büyür. Büyük blok daha geniş bölüm demektir, ancak küçük dosyalarda internal fragmentation (iç parçalanmayı) artırır: 32 kilobaytlık blokta saklanan 10 kilobaytlık dosya ayrılan yerin üçte ikisini boşa harcar. Bu file system (dosya sisteminde) inode bulunmaz; dosya adı, öznitelikler, tarih, ilk blok numarası ve boy 32 baytlık sabit uzunluklu dizin girdisinde durur. Bu nedenle aynı dosyaya ikinci bir ad bağlanamaz. Tablo bellekte tutulmak zorundadır ve boyu bölüm boyuyla birlikte büyür; buna karşılık yapı yalın ve gerçeklemesi kolaydır.

# 4

# Coalesced hashing (birleştirilmiş hashing)

Zincir bağlantısıyla collision resolution (çarpışma çözümleme)

NEDEN

## Yalnızca kendi zincirini izlemek

Open addressing (açık adreslemede) arama başka kayıtların overflow (taşmasından) da geçer

- Başarısız arama ancak boş yuvada biter

Link field (bağlantı alanı): aynı home address (ev adresinden) çıkan kayıtlar bir zincirde

- Arama yalnızca kendi zincirinde ilerler

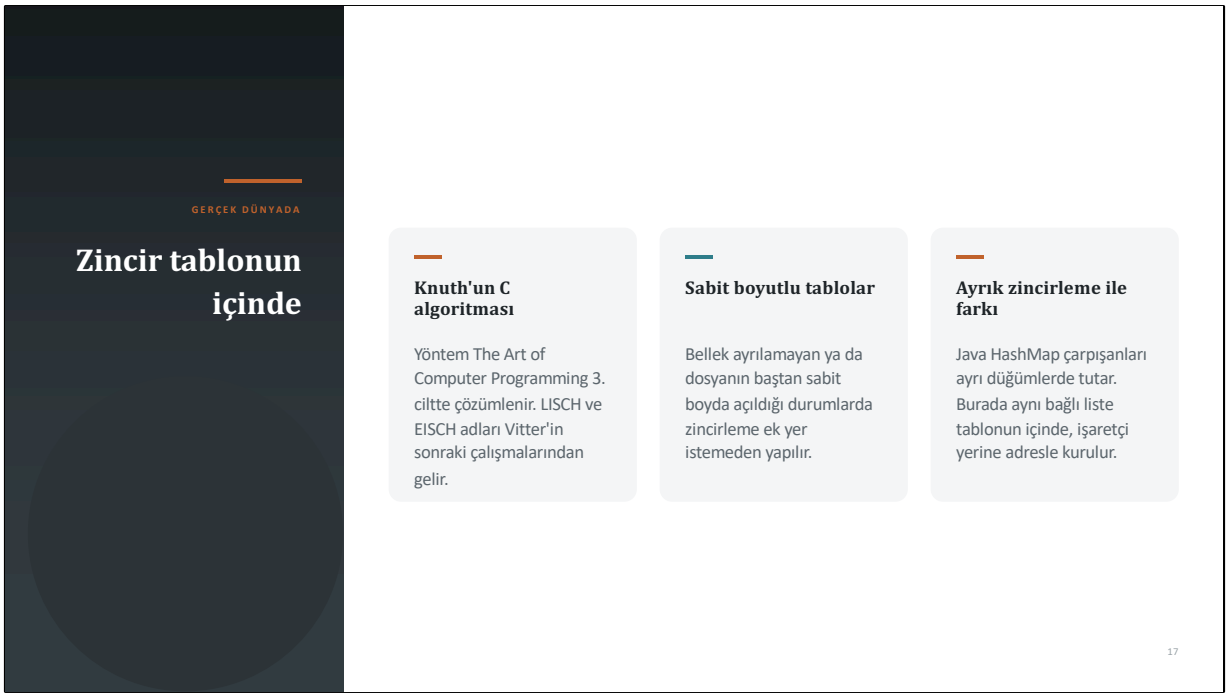
Zincir tablonun içinde kalır, ayrı bellek ayrılmaz

- Zincirler birbirine eklenebilir: birleştirilmiş (coalesced)

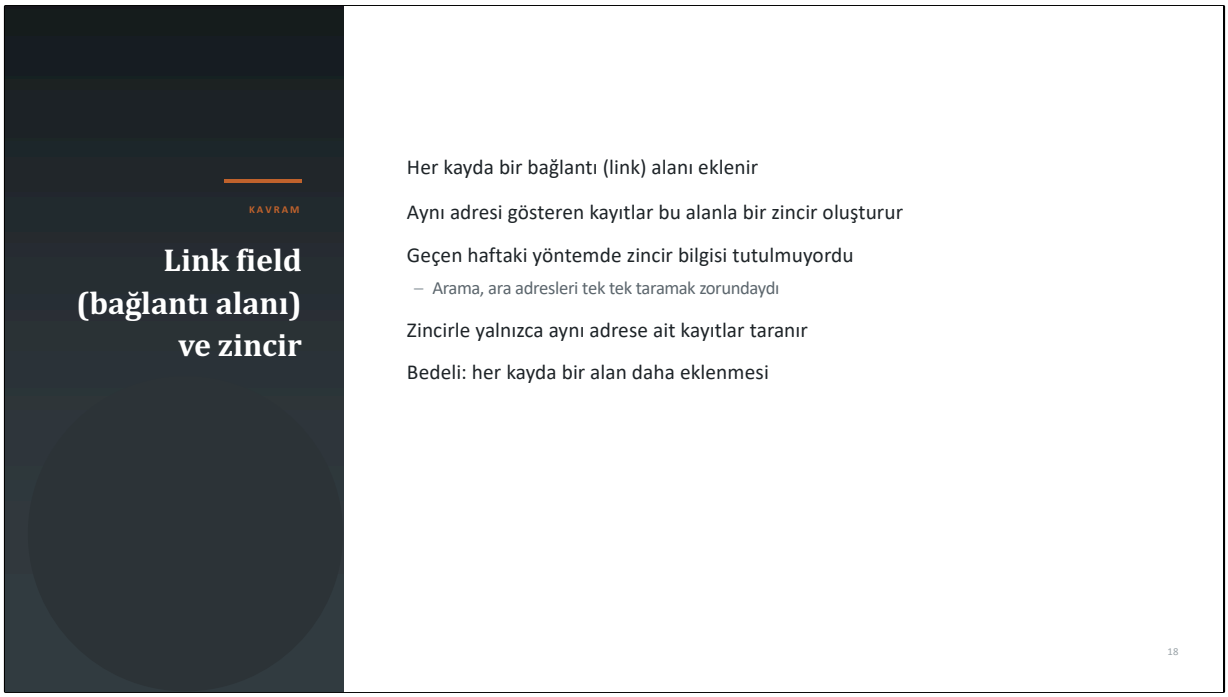
Bedel: kayıt başına bir bağlantı alanı



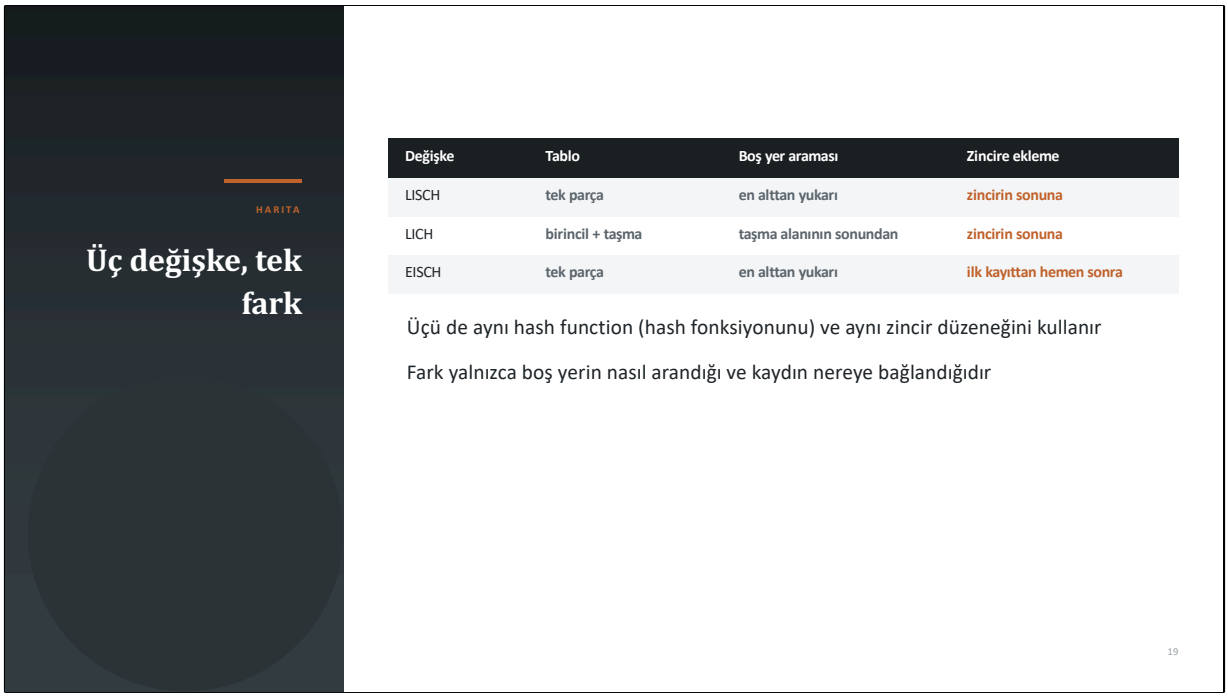
Open addressing (açık adresleme) yöntemlerinde bir kaydın aranması, başka home address (ev adreslerinden) taşmış kayıtların arasından da geçmeyi gerektirir; başarısız bir arama ancak boş bir yuvaya ulaşıldığında sona erer. Coalesced hashing (birleştirilmiş hashing) bu durumu bir link field (bağlantı alanıyla) çözer. Aynı ev adresinden çıkan kayıtlar bir zincir oluşturur ve arama yalnızca bu zincir boyunca ilerler. Zincir tablonun içinde kurulduğundan ayrı bir bellek alanı ayrılmaz. Farklı ev adreslerinden başlayan zincirler tablonun içinde birbirine eklenebilir; yöntemin adı bu birleşmeden gelmektedir. Kazancın karşılığı her kayda eklenen bağlantı alanıdır.




Coalesced hashing (birleştirilmiş hashing), Knuth'un The Art of Computer Programming kitabının üçüncü cildinde C algoritması olarak çözümlenmiştir. LISCH, EISCH ve LICH adları ise Vitter'in yöntemi ayrıntılı biçimde inceleyen sonraki çalışmalarından gelmektedir. Yöntem, çalışma sırasında bellek ayrılamayan ortamlarda ve dosyanın baştan sabit boyda açıldığı durumlarda zincirlemenin ek yer gerektirmeden yapılmasını sağlar. Java'nın HashMap yapısı çarpışan kayıtları tablonun dışında, ayrı düğümlerde tutar; buna ayrık zincirleme denir. Birleştirilmiş hashing aynı bağlı listeyi tablonun içinde, işaretçi yerine adres kullanarak kurar.




Collision (çarpışmaya) neden olan kayıtlar için bir link field (bağlantı alanı) oluşturulur ve kayıtların bağlantı bilgisi bu alanda tutulur. Aynı adresi gösteren kayıtlar bu bağlantı ile bir zincir oluşturur. Coalesced hashing (birleştirilmiş hashing) adı verilen bu yöntemin çeşitli değişkenleri vardır. Aralarındaki fark yalnızca tabloda boş yerin nasıl arandığı ve yeni kaydın zincire nereden eklendiğidir. Bedeli, her kayda bir bağlantı alanı eklenmesidir.




Üç değişken ele alınmaktadır. Birincisinde tablo bir bütündür ve boş yer en alttan aranır, yeni kayıt zincirin sonuna eklenir. İkincisinde tablo primary area (birincil alan) ve overflow area (taşma alanı) olarak ikiye ayrılır. Boş yer yalnızca taşma alanında aranır. Üçüncüsünde tablo yine bir bütündür, ancak yeni kayıt zincirin sonuna değil ilk kaydından hemen sonrasına eklenir. Üçü de aynı hash function (hash fonksiyonunu) ve aynı zincir düzeneğini kullanır.

ALGORITMA · LISCH

## Kayıt ekleme

Key (anahtara) hash function (hash fonksiyonu) uygulanır: home address (ev adresi)

Ev adresi boşsa kayıt oraya yazılır, bağlantıya ^ konur

Doluysa

- Kaydın zaten bulunup bulunmadığı denetlenir
- Tabloda en alttaki boş yer aranır
- Kayıt oraya yazılır, bağlantısına ^ konur
- Zincirin sonundaki kaydın bağlantısı yeni adrese ayarlanır

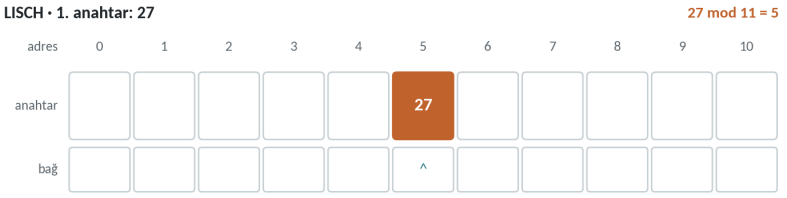
Boş yer kalmamışsa tablonun dolduğu bildirilir



Birinci değişkede ekleme şöyle yürür: key (anahtara) hash function (hash fonksiyonu) uygulanarak home address (ev adresi) bulunur. Ev adresi boşsa kayıt oraya yazılır ve link field (bağlantı alanına) zincir sonu işareti konur. Ev adresi doluysa önce kaydın zaten bulunup bulunmadığı denetlenir. Ardından tabloda en alttaki boş yer aranır. Kayıt bu yere yazılır, zincirin sonundaki kaydın bağlantı alanı yeni kaydın adresine ayarlanır. Boş yer kalmamışsa tablonun dolduğu bildirilir.

ADIM 1 / 9

## LISCH: 27 ekleniyor




27 = 11 × 2 + 5

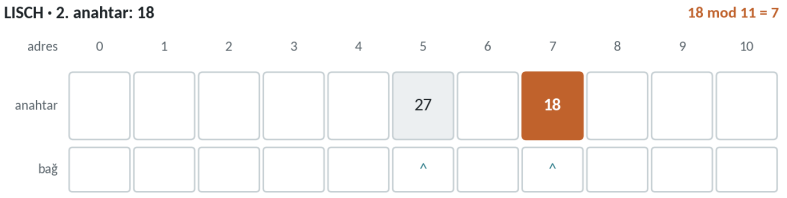
Home address (ev adresi) 5 boş: kayıt oraya, bağlantı ^

21

Birinci adımda 27 anahtarı eklenmektedir. Anahtarın home address (ev adresi) on bire bölümünden kalan olan 5 değeridir. Ev adresi boş olduğundan kayıt doğrudan buraya yazılmakta ve link field (bağlantı alanına) zincir sonu işareti konmaktadır. Bu adımdan sonra tabloda bir kayıt bulunmakta, doluluk oranı yüzde 9 olmaktadır. Boş adres sayısı azaldıkça sonraki anahtarların ev adresini dolu bulma olasılığı artmaktadır.

ADIM 2 / 9

## LISCH: 18 ekleniyor




18 = 11 × 1 + 7

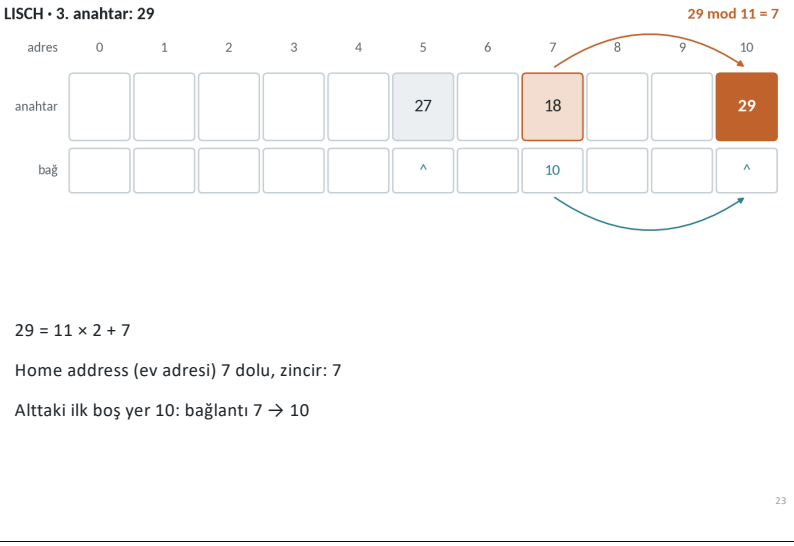
Home address (ev adresi) 7 boş: kayıt oraya, bağlantı ^

22

İkinci adımda 18 anahtarı eklenmektedir. Anahtarın home address (ev adresi) on bire bölümünden kalan olan 7 değeridir. Ev adresi boş olduğundan kayıt doğrudan buraya yazılmakta ve link field (bağlantı alanına) zincir sonu işareti konmaktadır. Bu adımdan sonra tabloda iki kayıt bulunmakta, doluluk oranı yüzde 18 olmaktadır. Boş adres sayısı azaldıkça sonraki anahtarların ev adresini dolu bulma olasılığı artmaktadır.

ADIM 3 / 9

## LISCH: 29 ekleniyor




29 = 11 × 2 + 7

Home address (ev adresi) 7 dolu, zincir: 7

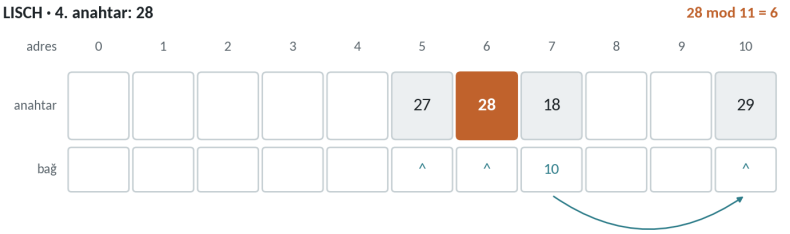
Alttaki ilk boş yer 10: bağlantı 7 → 10

23

Üçüncü adımda 29 anahtarı eklenmektedir. Anahtarın home address (ev adresi) on bire bölümünden kalan olan 7 değeridir. Ev adresi doludur. Önce zincir sonuna kadar ilerlenmekte, zincirin son kaydı 7 adresinde bulunmaktadır. Ardından tablonun en altından yukarı doğru boş yer aranmakta ve ilk boş yer olarak 10 adresi bulunmaktadır. Kayıt bu adrese yazılmakta ve 7 adresindeki kaydın link field (bağlantı alanı) 10 değerine ayarlanmaktadır.

ADIM 4 / 9

## LISCH: 28 ekleniyor




28 = 11 × 2 + 6

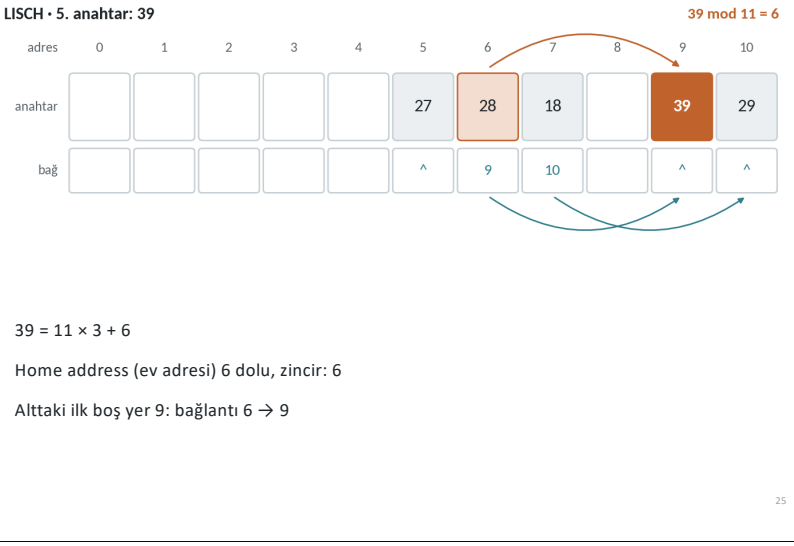
Home address (ev adresi) 6 boş: kayıt oraya, bağlantı ^

24

Dörtüncü adımda 28 anahtarı eklenmektedir. Anahtarın home address (ev adresi) on bire bölümünden kalan olan 6 değeridir. Ev adresi boş olduğundan kayıt doğrudan buraya yazılmakta ve link field (bağlantı alanına) zincir sonu işareti konmaktadır. Bu adımdan sonra tabloda dört kayıt bulunmakta, doluluk oranı yüzde 36 olmaktadır. Boş adres sayısı azaldıkça sonraki anahtarların ev adresini dolu bulma olasılığı artmaktadır.

ADIM 5 / 9

## LISCH: 39 ekleniyor




39 = 11 × 3 + 6

Home address (ev adresi) 6 dolu, zincir: 6

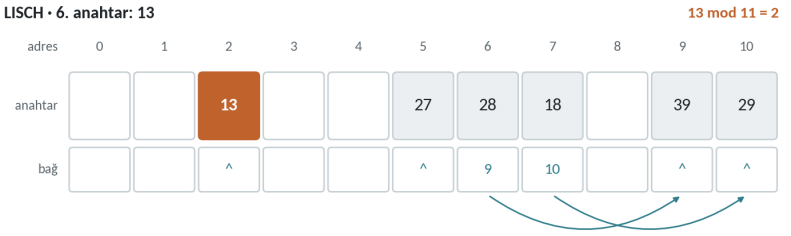
Alttaki ilk boş yer 9: bağlantı 6 → 9

25

Beşinci adımda 39 anahtarı eklenmektedir. Anahtarın home address (ev adresi) on bire bölümünden kalan olan 6 değeridir. Ev adresi doludur. Önce zincir sonuna kadar ilerlenmekte, zincirin son kaydı 6 adresinde bulunmaktadır. Ardından tablonun en altından yukarı doğru boş yer aranmakta ve ilk boş yer olarak 9 adresi bulunmaktadır. Kayıt bu adrese yazılmakta ve 6 adresindeki kaydın link field (bağlantı alanı) 9 değerine ayarlanmaktadır.

ADIM 6 / 9

## LISCH: 13 ekleniyor




13 = 11 × 1 + 2

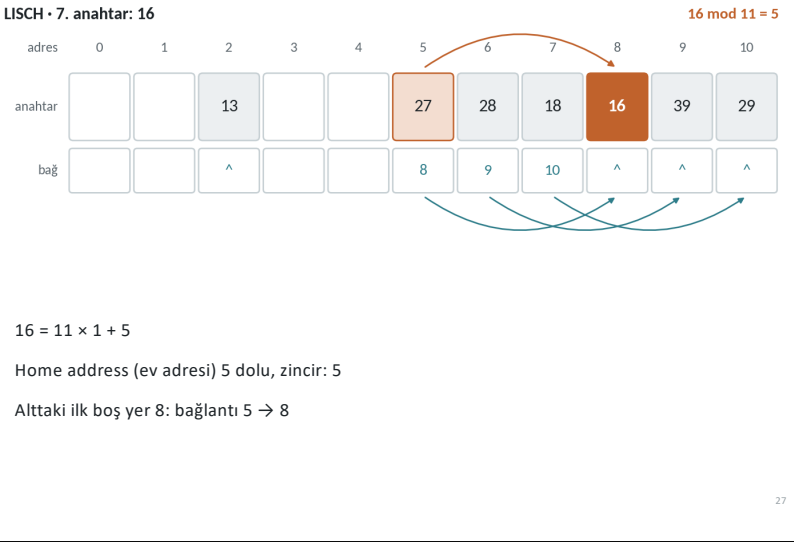
Home address (ev adresi) 2 boş: kayıt oraya, bağlantı ^

26

Altıncı adımda 13 anahtarı eklenmektedir. Anahtarın home address (ev adresi) on bire bölümünden kalan olan 2 değeridir. Ev adresi boş olduğundan kayıt doğrudan buraya yazılmakta ve link field (bağlantı alanına) zincir sonu işareti konmaktadır. Bu adımdan sonra tabloda altı kayıt bulunmakta, doluluk oranı yüzde 55 olmaktadır. Boş adres sayısı azaldıkça sonraki anahtarların ev adresini dolu bulma olasılığı artmaktadır.

ADIM 7 / 9

## LISCH: 16 ekleniyor




16 = 11 × 1 + 5

Home address (ev adresi) 5 dolu, zincir: 5

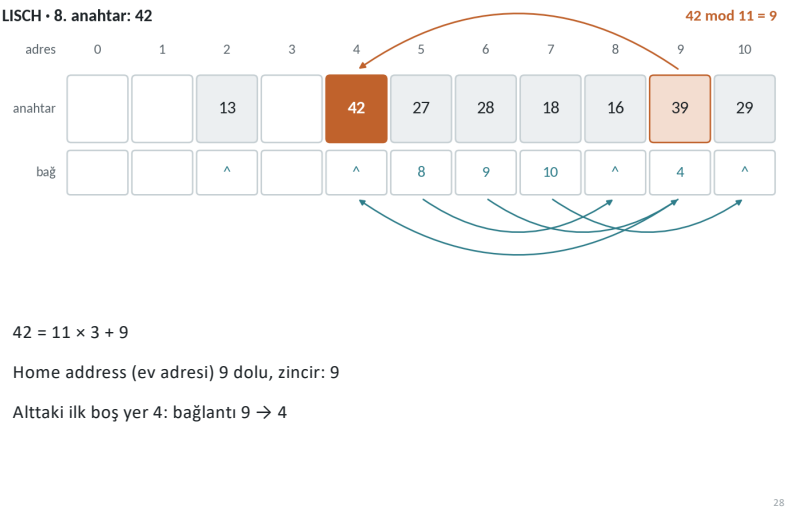
Alttaki ilk boş yer 8: bağlantı 5 → 8

27

Yedinci adımda 16 anahtarı eklenmektedir. Anahtarın home address (ev adresi) on bire bölümünden kalan olan 5 değeridir. Ev adresi doludur. Önce zincir sonuna kadar ilerlenmekte, zincirin son kaydı 5 adresinde bulunmaktadır. Ardından tablonun en altından yukarı doğru boş yer aranmakta ve ilk boş yer olarak 8 adresi bulunmaktadır. Kayıt bu adrese yazılmakta ve 5 adresindeki kaydın link field (bağlantı alanı) 8 değerine ayarlanmaktadır.

ADIM 8 / 9

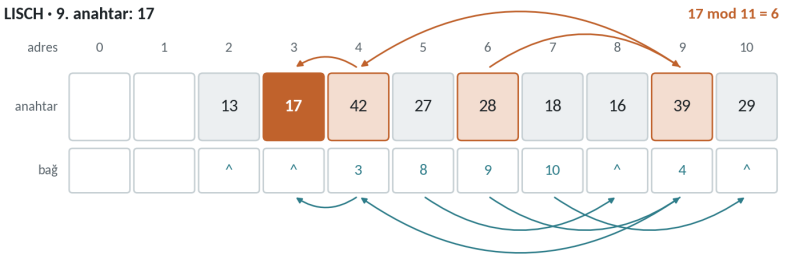
## LISCH: 42 ekleniyor




Sekizinci adımda 42 anahtarı eklenmektedir. Anahtarın home address (ev adresi) on bire bölümünden kalan olan 9 değeridir. Ev adresi doludur. Önce zincir sonuna kadar ilerlenmekte, zincirin son kaydı 9 adresinde bulunmaktadır. Ardından tablonun en altından yukarı doğru boş yer aranmakta ve ilk boş yer olarak 4 adresi bulunmaktadır. Kayıt bu adrese yazılmakta ve 9 adresindeki kaydın link field (bağlantı alanı) 4 değerine ayarlanmaktadır. Burada dikkat edilmesi gereken nokta, 9 adresindeki kaydın bu adrese ait olmamasıdır. Bu kayıt başka bir zincirden taşmıştır. Yeni kayıt o zincirin sonuna eklendiğinden iki zincir birleşmektedir.

ADIM 9 / 9

# LISCH: 17 ekleniyor




17 = 11 × 1 + 6

Home address (ev adresi) 6 dolu, zincir: 6 → 9 → 4

Alttaki ilk boş yer 3: bağlantı 4 → 3

29

Dokuzuncu adımda 17 anahtarı eklenmektedir. Anahtarın home address (ev adresi) on bire bölümünden kalan olan 6 değeridir. Ev adresi doludur. Önce zincir sonuna kadar ilerlenmekte, zincirin son kaydı 4 adresinde bulunmaktadır. Ardından tablonun en altından yukarı doğru boş yer aranmakta ve ilk boş yer olarak 3 adresi bulunmaktadır. Kayıt bu adrese yazılmakta ve 4 adresindeki kaydın link field (bağlantı alanı) 3 değerine ayarlanmaktadır.

ÖZET

## LISCH · adım adım

H(k) = k mod 11 · 27 · 18 · 29 · 28 · 39 · 13 · 16 · 42 · 17

| Anahtar | Ev adresi | Durum | Yerleşti | Bağlantı kurulan |
|---|---|---|---|---|
| 27 | 5 | boş | 5 | : |
| 18 | 7 | boş | 7 | : |
| 29 | 7 | dolu | 10 | 18 → 10 |
| 28 | 6 | boş | 6 | : |
| 39 | 6 | dolu | 9 | 28 → 9 |
| 13 | 2 | boş | 2 | : |
| 16 | 5 | dolu | 8 | 27 → 8 |
| 42 | 9 | dolu: başka zincir | 4 | 39 → 4 |
| 17 | 6 | dolu | 3 | 42 → 3 |

42'nin home address (ev adresi) başka bir zincirin kaydıyla dolu: iki zincir birleşiyor



Tabloda dokuz anahtarın adım adım yerleşimi verilmektedir. İlk dört anahtar home address (ev adreslerine) sorunsuz yerleşir. Beşinci anahtardan başlayarak collision (çarpışmalar) görülür ve taşan kayıtlar tablonun altından doldurulur. Dikkat çekici olan sekizinci adımdır: 42'nin ev adresi olan 9, başka bir zincire ait bir kayıtla doludur. Kayıt yine de o zincirin sonuna eklenir ve iki zincir birleşir. Yöntemin adındaki birleştirme sözcüğü bu durumu anlatmaktadır.

SONUÇ

## LISCH tablosu

| | Kayıt | Bağlantı | |
|---|---|---|---|
| 0 | | | |
| 1 | | | |
| 2 | 13 | ^ | |
| 3 | 17 | ^ | 6 → 9 → 4 → 3 |
| 4 | 42 | 3 | 28 → 39 → 42 → 17 |
| 5 | 27 | 8 | |
| 6 | 28 | 9 | 5 → 8 |
| 7 | 18 | 10 | 27 → 16 |
| 8 | 16 | ^ | |
| 9 | 39 | 4 | 7 → 10 |
| 10 | 29 | ^ | 18 → 29 |

Üç zincir, en uzunu 6 numaralı adresten başlıyor

Son kayda erişim dört probe gerektiriyor



Sonuç tablosunda üç zincir bulunmaktadır. En uzun zincir 6 numaralı adresten başlamakta ve dört kayıt taşımaktadır. Bu zincirin son kaydına erişmek dört probe gerektirir. Zincirlerin birleşmesi, bir adrese ait olmayan kayıtların da o zincirde taranmasına yol açar. Link field (bağlantı alanının) getirdiği kazanç, progressive overflow (adım adım ilerleyen taşmada) olduğu gibi bütün ara adreslerin taranmasının gerekmemesidir.

ALGORITMA · LICH

# Primary area (birincil alan) ve overflow area (taşma alanı)

Tablo ikiye ayrılır

- Home address (ev adresleri) yalnızca birincil alana düşer
- Hash function (hash fonksiyonunun) bölücüsü bu alanın boyuna göre seçilir

Collision (çarpışmada) boş yer, taşma alanının en büyük adresinden geriye aranır

Amaç: taşan kayıtların birincil alandaki adresleri işgal etmemesi

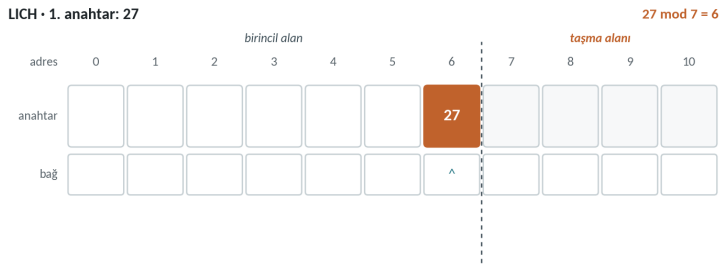
- Sonradan gelen ve ev adresi o adres olan kayıt çarpışmaz



Tablo ikiye ayrılır. Home address (ev adresleri) yalnızca primary area (birincil alana) düşer. Hash function (hash fonksiyonunun) bölücüsü bu alanın boyuna göre seçilir. Collision (çarpışma) durumunda boş yer, overflow area (taşma alanının) en büyük adresinden geriye doğru aranır. Bu ayrım, taşan kayıtların birincil alandaki boş adresleri işgal etmesini önler. Böylece sonradan gelen ve ev adresi o adres olan bir kaydın çarpışmaya uğraması engellenmiş olur.

ADIM 1 / 11

# LICH: 27 ekleniyor




27 = 7 × 3 + 6

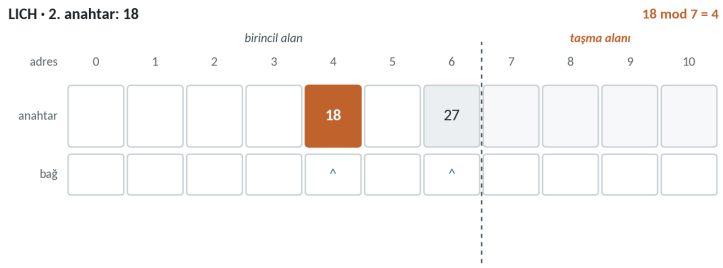
Home address (ev adresi) 6 boş: kayıt oraya

33

Birinci adımda 27 anahtarı eklenmektedir. Primary area (birincil alan) yedi adres olduğundan hash function (hash fonksiyonu) yediye bölümden kalandır ve home address (ev adresi) 6 olmaktadır. Ev adresi boş olduğundan kayıt birincil alandaki kendi adresine yazılmaktadır. Birincil alanın yedi adresinden bir tanesi bu adımdan sonra doludur. Overflow area (taşma alanı) yalnızca çarpışan kayıtlar için ayrıldığından henüz kullanılmamaktadır.

ADIM 2 / 11

## LICH: 18 ekleniyor




18 = 7 × 2 + 4

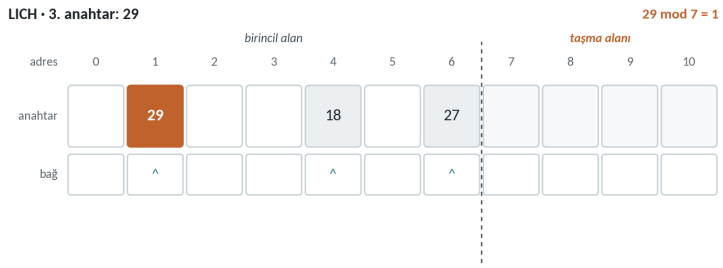
Home address (ev adresi) 4 boş: kayıt oraya

34

İkinci adımda 18 anahtarı eklenmektedir. Primary area (birincil alan) yedi adres olduğundan hash function (hash fonksiyonu) yediye bölümden kalandır ve home address (ev adresi) 4 olmaktadır. Ev adresi boş olduğundan kayıt birincil alandaki kendi adresine yazılmaktadır. Birincil alanın yedi adresinden iki tanesi bu adımdan sonra doludur. Overflow area (taşma alanı) yalnızca çarpışan kayıtlar için ayrıldığından henüz kullanılmamaktadır.

ADIM 3 / 11

## LICH: 29 ekleniyor




29 = 7 × 4 + 1

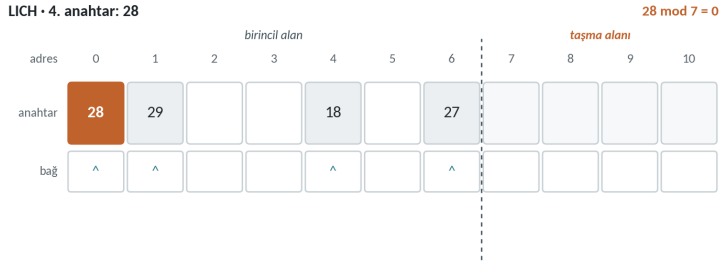
Home address (ev adresi) 1 boş: kayıt oraya

35

Üçüncü adımda 29 anahtarı eklenmektedir. Primary area (birincil alan) yedi adres olduğundan hash function (hash fonksiyonu) yediye bölümden kalandır ve home address (ev adresi) 1 olmaktadır. Ev adresi boş olduğundan kayıt birincil alandaki kendi adresine yazılmaktadır. Birincil alanın yedi adresinden üç tanesi bu adımdan sonra doludur. Overflow area (taşma alanı) yalnızca çarpışan kayıtlar için ayrıldığından henüz kullanılmamaktadır.

ADIM 4 / 11

## LICH: 28 ekleniyor




28 = 7 × 4 + 0

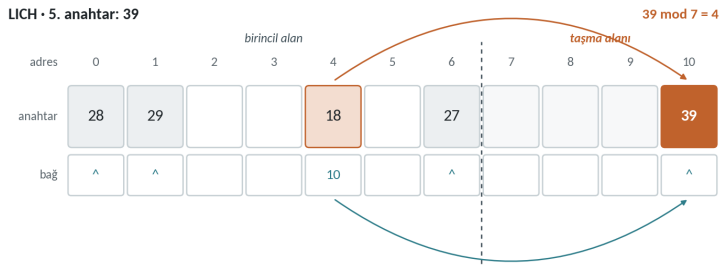
Home address (ev adresi) 0 boş: kayıt oraya

36

Dörtüncü adımda 28 anahtarı eklenmektedir. Primary area (birincil alan) yedi adres olduğundan hash function (hash fonksiyonu) yediye bölümden kalandır ve home address (ev adresi) 0 olmaktadır. Ev adresi boş olduğundan kayıt birincil alandaki kendi adresine yazılmaktadır. Birincil alanın yedi adresinden dört tanesi bu adımdan sonra doludur. Overflow area (taşma alanı) yalnızca çarpışan kayıtlar için ayrıldığından henüz kullanılmamaktadır.

ADIM 5 / 11

# LICH: 39 ekleniyor




39 = 7 × 5 + 4

Home address (ev adresi) 4 dolu: overflow area (taşma alanına)

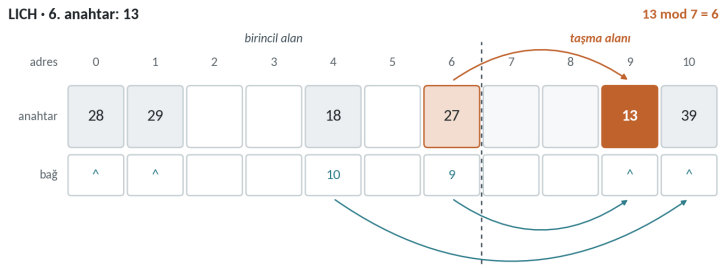
Taşma alanında ilk boş yer 10: bağlantı 4 → 10

37

Beşinci adımda 39 anahtarı eklenmektedir. Primary area (birincil alan) yedi adres olduğundan hash function (hash fonksiyonu) yediye bölümden kalandır ve home address (ev adresi) 4 olmaktadır. Ev adresi doludur. Boş yer birincil alanda değil, yalnızca overflow area (taşma alanında) ve en büyük adresten geriye doğru aranmaktadır. İlk boş yer 10 adresidir. Kayıt buraya yazılmakta ve zincirin son kaydının bağlantısı 10 değerine ayarlanmaktadır.

ADIM 6 / 11

## LICH: 13 ekleniyor




13 = 7 × 1 + 6

Home address (ev adresi) 6 dolu: overflow area (taşma alanına)

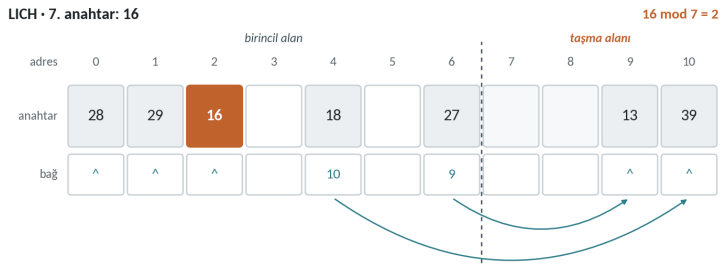
Taşma alanında ilk boş yer 9: bağlantı 6 → 9

38

Altıncı adımda 13 anahtarı eklenmektedir. Primary area (birincil alan) yedi adres olduğundan hash function (hash fonksiyonu) yediye bölümden kalandır ve home address (ev adresi) 6 olmaktadır. Ev adresi doludur. Boş yer birincil alanda değil, yalnızca overflow area (taşma alanında) ve en büyük adresten geriye doğru aranmaktadır. İlk boş yer 9 adresidir. Kayıt buraya yazılmakta ve zincirin son kaydının bağlantısı 9 değerine ayarlanmaktadır.

ADIM 7 / 11

# LICH: 16 ekleniyor




16 = 7 × 2 + 2

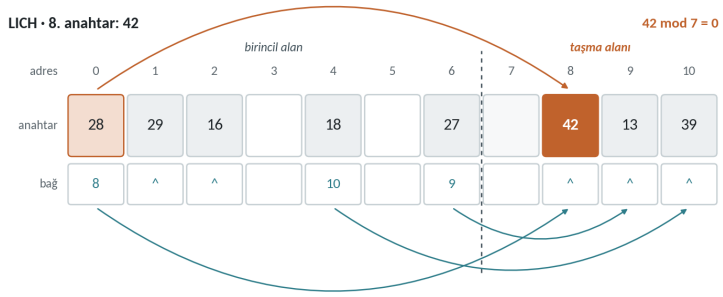
Home address (ev adresi) 2 boş: kayıt oraya

39

Yedinci adımda 16 anahtarı eklenmektedir. Primary area (birincil alan) yedi adres olduğundan hash function (hash fonksiyonu) yediye bölümden kalandır ve home address (ev adresi) 2 olmaktadır. Ev adresi boş olduğundan kayıt birincil alandaki kendi adresine yazılmaktadır. Birincil alanın yedi adresinden beş tanesi bu adımdan sonra doludur. Overflow area (taşma alanında) ise iki kayıt bulunmaktadır.

ADIM 8 / 11

# LICH: 42 ekleniyor




42 = 7 × 6 + 0

Home address (ev adresi) 0 dolu: overflow area (taşma alanına)

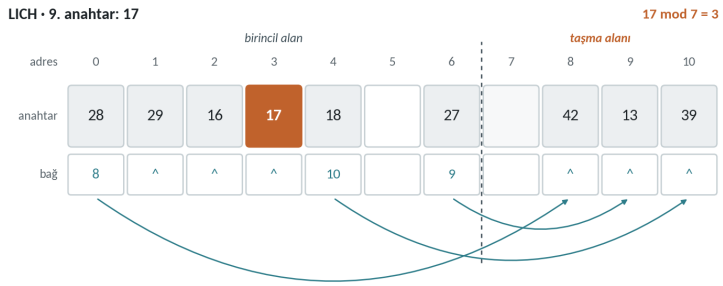
Taşma alanında ilk boş yer 8: bağlantı 0 → 8

40

Sekizinci adımda 42 anahtarı eklenmektedir. Primary area (birincil alan) yedi adres olduğundan hash function (hash fonksiyonu) yediye bölümden kalandır ve home address (ev adresi) 0 olmaktadır. Ev adresi doludur. Boş yer birincil alanda değil, yalnızca overflow area (taşma alanında) ve en büyük adresten geriye doğru aranmaktadır. İlk boş yer 8 adresidir. Kayıt buraya yazılmakta ve zincirin son kaydının bağlantısı 8 değerine ayarlanmaktadır.

ADIM 9 / 11

# LICH: 17 ekleniyor




17 = 7 × 2 + 3

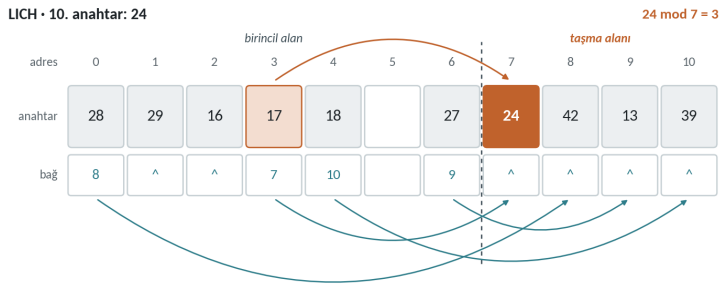
Home address (ev adresi) 3 boş: kayıt oraya

41

Dokuzuncu adımda 17 anahtarı eklenmektedir. Primary area (birincil alan) yedi adres olduğundan hash function (hash fonksiyonu) yediye bölümden kalandır ve home address (ev adresi) 3 olmaktadır. Ev adresi boş olduğundan kayıt birincil alandaki kendi adresine yazılmaktadır. Birincil alanın yedi adresinden altı tanesi bu adımdan sonra doludur. Overflow area (taşma alanında) ise üç kayıt bulunmaktadır.

ADIM 10 / 11

# LICH: 24 ekleniyor




24 = 7 × 3 + 3

Home address (ev adresi) 3 dolu: overflow area (taşma alanına)

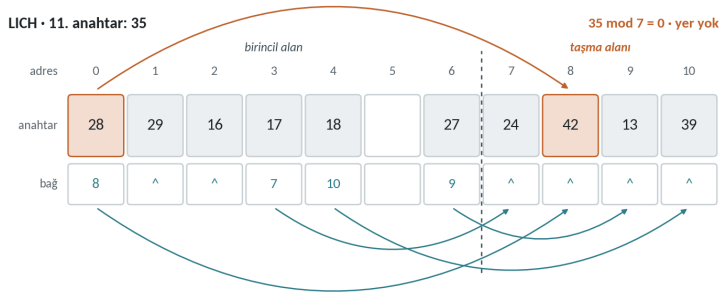
Taşma alanında ilk boş yer 7: bağlantı 3 → 7

42

Onuncu adımda 24 anahtarı eklenmektedir. Primary area (birincil alan) yedi adres olduğundan hash function (hash fonksiyonu) yediye bölümden kalandır ve home address (ev adresi) 3 olmaktadır. Ev adresi doludur. Boş yer birincil alanda değil, yalnızca overflow area (taşma alanında) ve en büyük adresten geriye doğru aranmaktadır. İlk boş yer 7 adresidir. Kayıt buraya yazılmakta ve zincirin son kaydının bağlantısı 7 değerine ayarlanmaktadır.

ADIM 11 / 11

# LICH: 35 ekleniyor




35 = 7 × 5 + 0

Home address (ev adresi) 0 dolu, overflow area (taşma alanı) dolu

5 boş olmasına karşın ekleme başarısız

43

On birinci adımda 35 anahtarı eklenmektedir. Primary area (birincil alan) yedi adres olduğundan hash function (hash fonksiyonu) yediye bölümden kalandır ve home address (ev adresi) 0 olmaktadır. Ev adresi doludur ve overflow area (taşma alanının) dört adresi de önceki collision (çarpışmalarla) dolmuştur. Birincil alanda beşinci adres boş olmasına karşın bu adres yalnızca ev adresi beş olan kayıtlar içindir. Yöntem bu nedenle kaydı yerleştirememekte ve tablonun dolduğunu bildirmektedir. İki alanın bağımsız dolması budur.

ÖZET

## LICH tablosu

| | Kayıt | Bağlantı | |
|---|---|---|---|
| 0 | 28 | 8 | |
| 1 | 29 | ^ | |
| 2 | 16 | ^ | |
| 3 | 17 | 7 | Birincil alan 0 – 6 |
| 4 | 18 | 10 | |
| 5 | | | |
| 6 | 27 | 9 | |
| 7 | 24 | ^ | |
| 8 | 42 | ^ | Taşma alanı 7 – 10 |
| 9 | 13 | ^ | |
| 10 | 39 | ^ | |

5 boş — ama taşma alanı dolu: 35 eklenemiyor

H(k) = k mod 7 · primary area (birincil alan) 0 ile 6 · overflow area (taşma alanı) 7 ile 10

44

Örnekte on anahtar, yedi adreslik primary area (birincil alanı) ve dört adreslik overflow area (taşma alanı) bulunan bir tabloya yerleştirilmektedir. Hash function (hash fonksiyonu) yediye bölümden kalandır, çünkü home address (ev adresleri) yalnızca birincil alana düşmelidir. Çarpışan dört kayıt, taşma alanının en büyük adresinden başlanarak onuncu, dokuzuncu, sekizinci ve yedinci adreslere yerleşmektedir. Taşan kayıtlar birincil alandaki hiçbir adresi işgal etmediğinden, ev adresi beşinci adres olan bir kayıt geldiğinde collision (çarpışma) yaşamayacaktır. Buna karşılık taşma alanı dolduğu anda, ev adresi dolu olan yeni bir kayıt yerleştirilemez duruma gelmektedir.

SINIR

## İki alan bağımsız dolar

Örnekte overflow area (taşma alanının) dört adresi de doldu

Yeni bir çarpışan kayıt yerleştirilemiyor

- Oysa primary area (birincil alanda) 5 numaralı adres boş

Tablo yüzde doksan dolu, ama ekleme başarısız

İki alanın göreli boyu bu nedenle bir tasarım kararıdır

- Beklenen collision (çarpışma) sayısına göre belirlenmelidir



Ayrımın bedeli, iki alanın birbirinden bağımsız dolmasıdır. Örnekte overflow area (taşma alanının) dört adresi de dolduğunda yeni bir çarpışan kayıt yerleştirilememektedir. Oysa primary area (birincil alanda) hâlâ boş bir adres bulunmaktadır. Tablo bütün olarak yüzde doksan dolu olmasına karşın ekleme başarısız olur. İki alanın göreli boyu bu nedenle tasarım kararıdır ve beklenen collision (çarpışma) sayısına göre belirlenmelidir.

ALIŞTIRMA 1 · SORU

## LICH

**H(k) = k mod 7 · birincil 0 – 6 · taşma 7 – 10**

Collision (çarpışmada) boş yer taşma alanının en büyük adresinden geriye aranır

Yeni kayıt zincirin sonuna bağlanır; taşma alanı dolarsa ekleme yapılamaz

ANAHTARLAR · BU SIRAYLA EKLENİR

90 80 35 97 59 71 87 20 63

| Anahtar | Ev adresi | Yer | Zincir | Bağlantı | Probe |
|---|---|---|---|---|---|
| 90 | ? | ? | ? | ? | ? |
| 80 | ? | ? | ? | ? | ? |
| 35 | ? | ? | ? | ? | ? |
| 97 | ? | ? | ? | ? | ? |
| 59 | ? | ? | ? | ? | ? |
| 71 | ? | ? | ? | ? | ? |
| 87 | ? | ? | ? | ? | ? |
| 20 | ? | ? | ? | ? | ? |
| 63 | ? | ? | ? | ? | ? |

46

Bu alıştırmada tablo, yedi gözlük primary area (birincil alan) ve dört gözlük overflow area (taşma alanı) olarak ikiye ayrılmıştır. Hash function (hash fonksiyonu) yediye bölümden kalan olduğundan ev adresleri yalnızca sıfır ile altı arasına düşer. Öğrencilerden dokuz anahtarı sırayla yerleştirmeleri, her anahtarın ev adresinden başlayan zinciri yazmaları ve son durumdaki link field (bağlantı alanı) değeri ile probe sayısını bulmaları beklenmektedir. Son anahtarın yerleştirilip yerleştirilemeyeceği ayrıca tartışılmalıdır: birincil alanda boş göz bulunmasının ekleme için yeterli olup olmadığı bu soruyla sınanmaktadır.

ALIŞTIRMA 1 · ÇÖZÜM

# LICH

H(k) = k mod 7 · birincil 0 – 6 · taşma 7 – 10

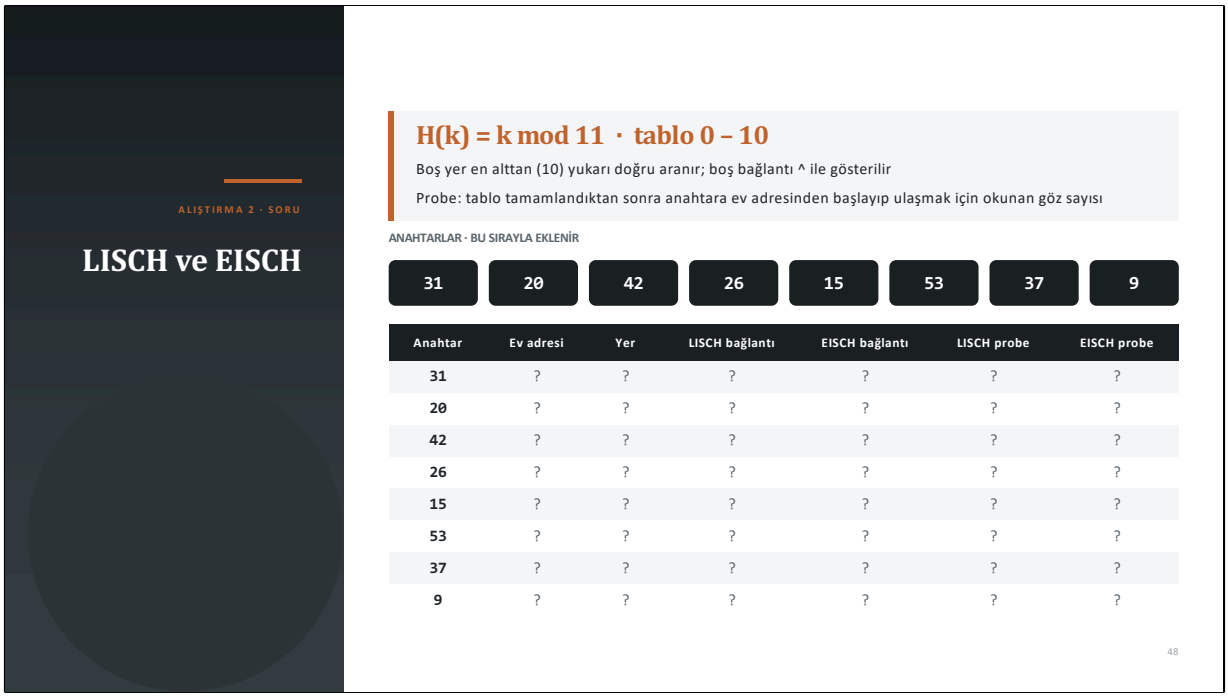
| Anahtar | Ev adresi | Yer | Zincir | Bağlantı | Probe |
|---|---|---|---|---|---|
| **90** | 6 | **6** | 6 | 10 | **1** |
| **80** | 3 | **3** | 3 | 9 | **1** |
| **35** | 0 | **0** | 0 | ^ | **1** |
| **97** | 6 | **10** | 6 → 10 | 7 | **2** |
| **59** | 3 | **9** | 3 → 9 | 8 | **2** |
| **71** | 1 | **1** | 1 | ^ | **1** |
| **87** | 3 | **8** | 3 → 9 → 8 | ^ | **3** |
| **20** | 6 | **7** | 6 → 10 → 7 | ^ | **3** |
| **63** | 0 | – | taşma alanı dolu | – | – |

Toplam 14 probe · ortalama 14 / 8 = 1,75

63 eklenemiyor: 2, 4 ve 5 boş, ama yalnızca kendi ev adresleri için

47

Çözüm slaytında her tıklama bir anahtarın satırını getirmektedir. 90, 80 ve 35 anahtarları ev adreslerine yerleşir. 97 altıncı adreste, 59 üçüncü adreste çarpışır ve taşma alanının onuncu ve dokuzuncu gözlerine yazılır. 71 ev adresine yerleşir. 87 yine üçüncü adreste çarpışır; zincir 3 → 9 olduğundan sekizinci göze yazılır ve dokuzuncu gözün bağlantısı sekize ayarlanır. 20 altıncı adreste çarpışır ve 6 → 10 zincirinin sonuna, yedinci göze bağlanır. 63 sıfırıncı adreste çarpıştığında taşma alanında boş göz kalmadığı için eklenemez; birincil alandaki ikinci, dördüncü ve beşinci gözlerin boş olması sonucu değiştirmez. Sekiz kayıt için toplam on dört probe yapılmaktadır.




Hafta boyunca izlenen iki değişke bu alıştırmada aynı anahtar kümesiyle yeniden çalıştırılmaktadır. Öğrencilerden sekiz anahtarı on bir gözlü tabloya önce LISCH, sonra EISCH ile yerleştirmeleri beklenmektedir. Her iki yöntemde de collision (çarpışma) durumunda boş yer tablonun en alttaki gözünden yukarı doğru aranmaktadır; bu nedenle kayıtların yerleri iki yöntemde aynı çıkacaktır. Farkın ortaya çıktığı yer link field (bağlantı alanı) değerleri ve bunlara bağlı probe sayılarıdır. Bağlantı ve probe sütunları, bütün anahtarlar yerleştikten sonraki son duruma göre doldurulmalıdır.

ALIŞTIRMA 2 · ÇÖZÜM

## LISCH ve EISCH

H(k) = k mod 11 · bütün anahtarlar yerleştikten sonraki durum

| Anahtar | Ev adresi | Yer | LISCH bağlantı | EISCH bağlantı | LISCH probe | EISCH probe |
|---|---|---|---|---|---|---|
| **31** | 9 | **9** | 10 | 3 | **1** | **1** |
| **20** | 9 | **10** | 8 | ^ | **2** | **5** |
| **42** | 9 | **8** | 6 | 10 | **3** | **4** |
| **26** | 4 | **4** | 7 | 5 | **1** | **1** |
| **15** | 4 | **7** | 5 | ^ | **2** | **3** |
| **53** | 9 | **6** | 3 | 8 | **4** | **3** |
| **37** | 4 | **5** | ^ | 7 | **3** | **2** |
| **9** | 9 | **3** | ^ | 6 | **5** | **2** |
| **Toplam** | | | | | **21** | **21** |

Yerleşim aynı, yalnızca bağlantılar farklı

Toplam 21 ve 21 · ortalama 2,62 ve 2,62

– Turuncu: EISCH'te probe sayısı değişen anahtarlar

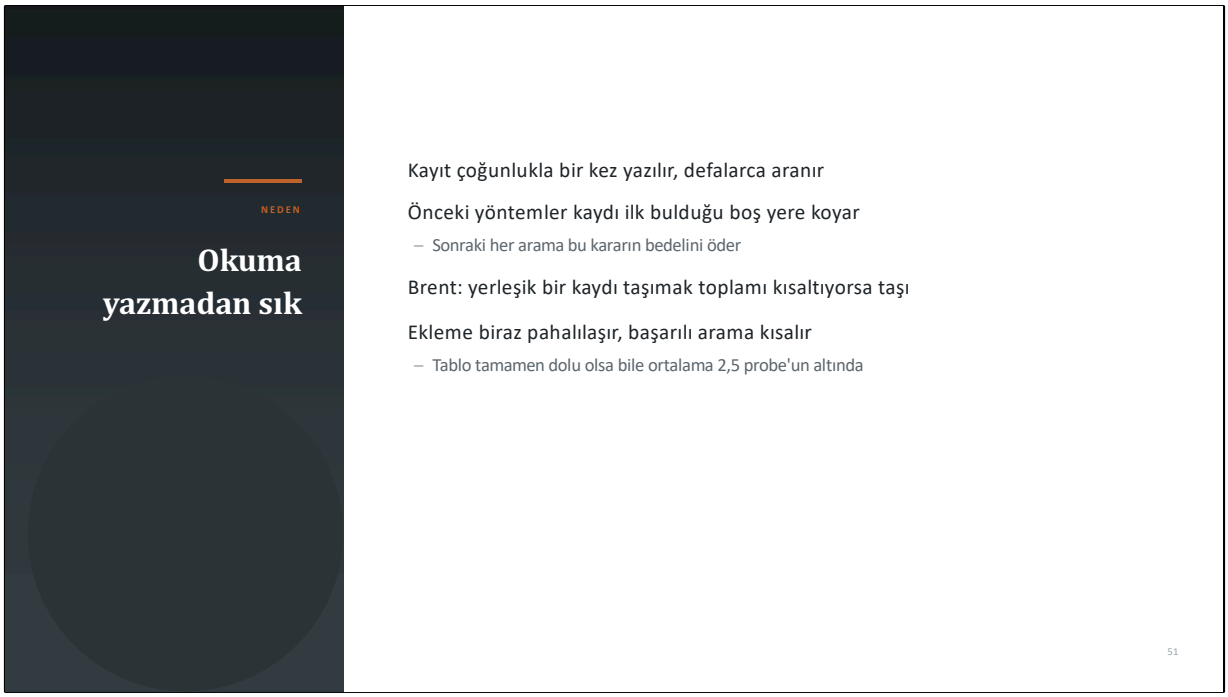
49

Sekiz anahtarın üçü home address (ev adresine) yerleşmekte, beşi collision (çarpışma) nedeniyle en alttan başlanarak onuncu, sekizinci, yedinci, altıncı ve beşinci gözlere yazılmaktadır. 31, 20, 42, 53 ve 9 anahtarları dokuzuncu adresten başlayan tek bir zincirde, 26, 15 ve 37 anahtarları dördüncü adresten başlayan ikinci zincirde toplanmaktadır. LISCH yeni kaydı zincirin sonuna, EISCH ilk kaydın hemen arkasına bağlamaktadır. Bu örnekte iki yöntemin toplam probe sayısı eşit çıkmıştır: EISCH son eklenen 9 ve 37 anahtarlarına erişimi kısaltırken erken eklenen 20 ve 42 anahtarlarına erişimi uzatmaktadır. Kazancın erişim sıklığının eklenme sırasıyla ilişkili olduğu durumlarda ortaya çıktığı bu tablodan görülebilir.

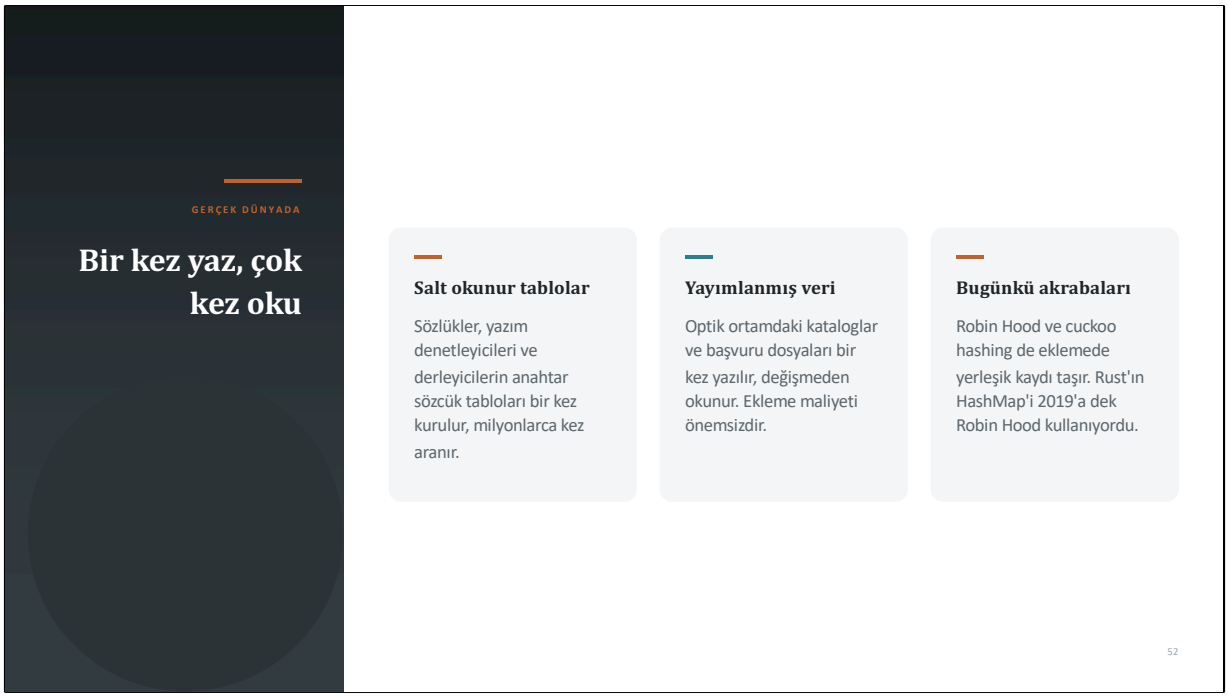
5

# Brent's metodu

Yerleşik kaydı taşıyan collision (çarpışma) çözümü




Çoğu dosyada bir kayıt bir kez yazılır ve ömrü boyunca defalarca aranır. Şimdiye kadar görülen yöntemlerde çarpışan kayıt ilk bulunan boş yere konur ve bu kararın bedelini o kayda yapılan her arama öder. Brent'in 1973'te önerdiği yöntem ekleme anında bir seçenek daha değerlendirir: yeni kaydı uzağa koymak yerine yolundaki yerleşik bir kaydı kendi ikincil zinciri üzerinde bir adım ileri taşımak toplam probe sayısını azaltıyorsa taşıma yapılır. Ekleme biraz pahalılaşır, buna karşılık başarılı arama kısalır. Knuth'un çözümlemesine göre tablo tamamen dolu olduğunda bile ortalama başarılı arama 2,5 probe'un altında kalır.




Yöntemin kazancı, aramanın eklemeden çok daha sık yapıldığı dosyalarda belirgindir. Sözlükler, yazım denetleyicileri ve derleyicilerin anahtar sözcük tabloları bir kez kurulur ve sonra çok sayıda kez aranır. Optik ortama yazılan kataloglar ve başvuru dosyaları da bir kez yazılıp değişmeden okunur; bu dosyalarda ekleme maliyeti pratikte önemsizdir. Ekleme sırasında yerleşik kaydı taşıma fikri bugün de kullanılmaktadır. Robin Hood hashing ve cuckoo hashing bu ailenin üyeleridir; Rust dilinin standart eşleme yapısı 2019 yılına kadar Robin Hood hashing kullanmaktaydı.

FIKIR

## Kayıtlar yer değiştirebilir

Şimdiye dek görülen bütün yöntemler durağandı

- Kayıtlar ilk nereye yerleştirilirse orada kalıyordu

Brent's yönteminde yerleşik anahtar sonradan taşınabilir

Yeni kaydı uzun bir zincirin sonuna atmak yerine

- Home address (ev adresindeki) kaydı taşı, yeni kaydı onun yerine koy

Amaç toplam probe sayısını düşürmek

Yer kararı eklemede bir kez değil, gerektikçe yeniden verilir



Şimdiye dek görülen bütün collision resolution (çarpışma çözümleme) yöntemleri durağandır: kayıtlar ilk nereye yerleştirilirse orada kalır. Brent's yönteminde ise başlangıçta bir adrese yerleştirilen anahtar daha sonra yer değiştirebilmektedir. Yöntem, yeni gelen kaydı uzun bir zincirin sonuna atmak yerine, home address (ev adresindeki) yerleşik kaydı taşıyıp yeni kaydı onun yerine koymayı değerlendirir. Amaç, toplam probe sayısını düşürmektir.

YAPI

# İki probe zinciri




i: yeni anahtarın birincil zincirindeki sıra (ev adresi i = 1)

j: o adresteki kaydın kendi artımıyla atacağı adım sayısı



Yöntemde iki ayrı probe zinciri izlenir. Birincil probe zinciri yeni kaydın ev adresinden başlar ve yeni kaydın ikinci hash değeri kadar artımla ilerler. i göstergesi bu zincir üzerindeki sırayı belirtir; ev adresi birinci sıradır. İkincil probe zinciri, birincil zincirin i. adresindeki yerleşik kaydın taşınabileceği adresleri verir ve o kaydın kendi ikinci hash değeri kadar artımla ilerler. j göstergesi bu zincirde atılan adım sayısını, dolayısıyla taşınan kaydın arama maliyetindeki artışı belirtir.

YAPI

## i, j ve s'nin hesaplanması

$$adres(i) = (H_1(k) + (i - 1) \cdot H_2(k)) \bmod M$$

hedef(j) = (adres(i) + j · $H_2$(r)) mod M · r: adres(i)'deki yerleşik kayıt

s: birincil zincirde ilk boş adresin sırası

Taşıma yoksa yeni kayıt s. probe'ta bulunur: maliyet s

Taşıma varsa yeni kayıt i. probe'ta bulunur, r'nin maliyeti j artar: i + j

Taşıma yalnızca i + j < s ise yapılır

Çiftler i + j toplamına göre küçükten büyüğe denenir, eşitlikte küçük i önce

İlk boş hedefte durulur

55

Brent's metodunda üç sayı kullanılmaktadır. s, yeni anahtarın birincil probe zincirinde ilk boş adresin kaçıncı sırada bulunduğunu gösterir; taşıma yapılmazsa yeni kayıt bu adrese yazılır ve s probe ile bulunur. i, birincil zincir üzerindeki sırayı belirtir; ev adresi birinci sıradır. j ise i. adresteki yerleşik kaydın kendi artımıyla, yani kendi $H_2$ değeriyle kaç adım ileri taşınacağını belirtir. Taşıma yapıldığında yeni kayıt i probe ile bulunur, taşınan kaydın arama maliyeti ise j kadar artar. Bu nedenle taşıma yalnızca i + j toplamı s değerinden küçükse kazançlıdır. Adaylar toplamı en küçük olandan başlanarak denenir ve ilk boş hedefte durulur.

ÖRNEK · I VE J

## 16 eklenirken

### 16: $H_1 = 5, H_2 = 1$

Birincil zincir 5 · 6 · 7 · 8 · 9 · 10: ilk boş adres 10, altıncı probe, s = 6

| i + j | i | adres(i) | r | $H_2(r)$ | j | hedef(j) | Durum |
|---|---|---|---|---|---|---|---|
| **2** | 1 | 5 | 27 | 2 | 1 | 7 | dolu (18) |
| **3** | 1 | 5 | 27 | 2 | 2 | 9 | dolu (29) |
| **3** | 2 | 6 | 39 | 3 | 1 | 9 | dolu (29) |
| **4** | 1 | 5 | 27 | 2 | 3 | 0 | **boş: taşı** |

i + j = 4 < s = 6: 27 adres 0'a taşınır, 16 ev adresi 5'e yazılır

Maliyet 6 yerine 4: 16 bir, 27 dört probe ile bulunur

56

Örnekte 16 anahtarının eklenmesi ele alınmaktadır. Ev adresi 5, artımı 1'dir. Birincil zincir 5, 6, 7, 8 ve 9 adreslerinden geçmekte ve ilk boş adres olan 10'a altıncı probe'ta ulaşmaktadır; dolayısıyla s altıdır. Adaylar i + j toplamına göre sıralanır. Toplamı iki ve üç olan çiftlerde hedef adresler doludur. Toplamı dört olan ilk çiftte 27 anahtarı kendi artımı olan 2 ile üç adım ilerleyerek boş olan 0 adresine ulaşmaktadır. Dört altıdan küçük olduğundan taşıma yapılır ve 16 ev adresine yazılır.

ALGORITMA · BRENT

# Kayıt ekleme

Anahtara birinci hash function (hash fonksiyonunu) uygula

Home address (ev adresi) boşsa kaydı buraya ekle, değilse

- Birincil zincirde ilk boş yeri ara, kaçıncı probe olduğunu s'de tut
- $i \leftarrow 1, j \leftarrow 1$

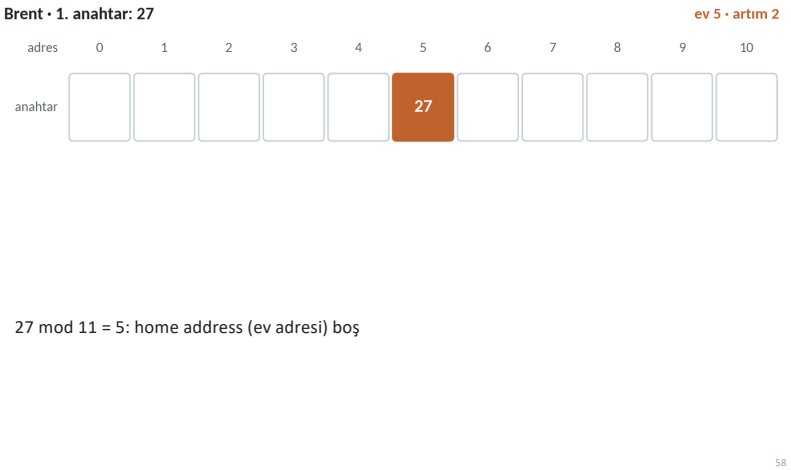
$(i + j) < s$ olduğu sürece taşıma dene

- Birincil zincirin i. adresindeki kayıt, kendi ikincil zincirinde j. adrese taşınabilir mi
- Taşınabiliyorsa taşı, yeni kaydı boşalan yere yaz
- Olmuyorsa $(i + j)$ toplamını büyüt, eşitlerde i'yi küçük al

Hiçbir çift işe yaramazsa kaydı birincil zincirdeki s. yere ekle

ADIM 1 / 9

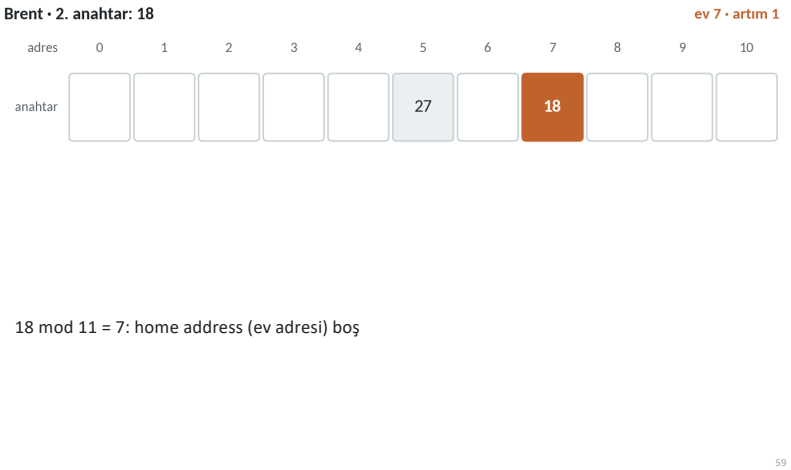
## Brent's metodu: 27




Birinci adımda 27 anahtarı eklenmektedir. Home address (ev adresi) on bire bölümden kalan olan 5, artım ise bölümün kendisi olan 2 değeridir. Ev adresi boş olduğundan kayıt doğrudan buraya yerleşmektedir ve hiçbir taşıma değerlendirilmemektedir. Bu adımdan sonra tabloda bir kayıt bulunmaktadır. Taşıma yalnızca ev adresi dolu olduğunda gündeme gelmektedir.

ADIM 2 / 9

## Brent's metodu: 18



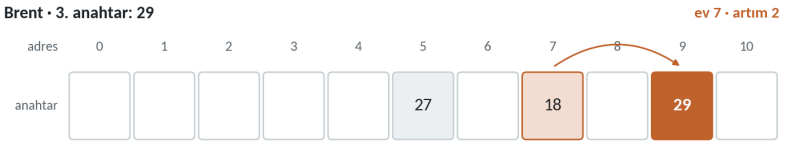

18 mod 11 = 7: home address (ev adresi) boş



İkinci adımda 18 anahtarı eklenmektedir. Home address (ev adresi) on bire bölümden kalan olan 7, artım ise bölümün kendisi olan 1 değeridir. Ev adresi boş olduğundan kayıt doğrudan buraya yerleşmektedir ve hiçbir taşıma değerlendirilmemektedir. Bu adımdan sonra tabloda iki kayıt bulunmaktadır. Taşıma yalnızca ev adresi dolu olduğunda gündeme gelmektedir.

ADIM 3 / 9

## Brent's metodu: 29




Home address (ev adresi) 7 dolu, artım 2

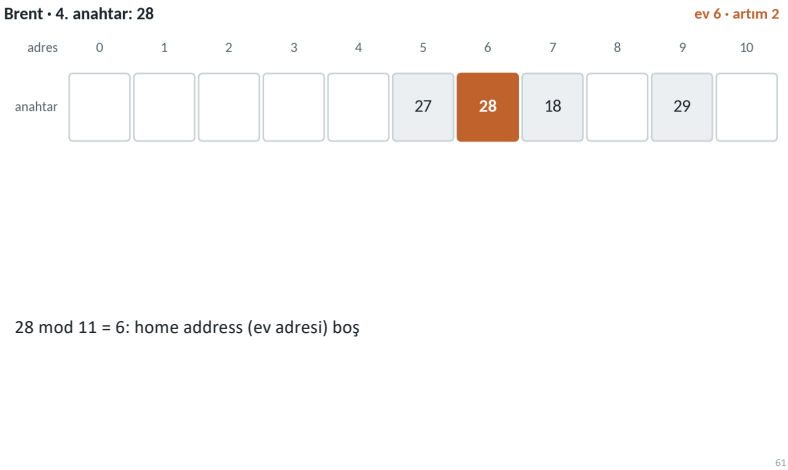
Birincil zincirde ilk boş yer 9: taşıma kazandırmaz

60

Üçüncü adımda 29 anahtarı eklenmektedir. Home address (ev adresi) on bire bölümden kalan olan 7, artım ise bölümün kendisi olan 2 değeridir. Ev adresi doludur. Birincil zincir üzerinde artım 2 ile ilerlendiğinde ilk boş yer 9 adresinde, ikinci probe'ta bulunmaktadır. s değeri iki olduğundan i ile j toplamının ikiden küçük olması olanaksızdır ve taşıma denenmemektedir. Kayıt birincil zincirdeki boş yere yazılmaktadır.

ADIM 4 / 9

## Brent's metodu: 28




Dörtüncü adımda 28 anahtarı eklenmektedir. Home address (ev adresi) on bire bölümden kalan olan 6, artım ise bölümün kendisi olan 2 değeridir. Ev adresi boş olduğundan kayıt doğrudan buraya yerleşmektedir ve hiçbir taşıma değerlendirilmemektedir. Bu adımdan sonra tabloda dört kayıt bulunmaktadır. Taşıma yalnızca ev adresi dolu olduğunda gündeme gelmektedir.

ADIM 5 / 9

## Brent's metodu: 39




Home address (ev adresi) 6 dolu: yerleşik 28 taşınabilir mi

28, artımı 2 ile 1 adım: 6 → 8 (i = 1, j = 1)

39 ev adresine yerleşir

62

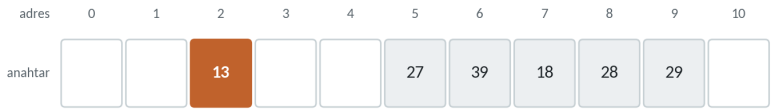
Beşinci adımda 39 anahtarı eklenmektedir. Home address (ev adresi) on bire bölümden kalan olan 6, artım ise bölümün kendisi olan 3 değeridir. Ev adresi 28 anahtarıyla doludur. Yeni kaydın birincil zincirinde boş yer daha uzakta olduğundan, yerleşik kaydın kendi ikincil zinciri üzerinde taşınması değerlendirilmektedir. 28 anahtarının artımı 2 olduğundan 1 adım sonra 8 adresine ulaşmaktadır ve bu adres boştur. i ile j toplamı birincil zincirde yeni kaydın yerleşeceği probe sayısından küçük olduğu için taşıma kazançlıdır. 28 taşınmakta ve 39 ev adresine yazılmaktadır.

ADIM 6 / 9

## Brent's metodu: 13

Brent · 6. anahtar: 13

ev 2 · artım 1




13 mod 11 = 2: home address (ev adresi) boş

63

Altıncı adımda 13 anahtarı eklenmektedir. Home address (ev adresi) on bire bölümden kalan olan 2, artım ise bölümün kendisi olan 1 değeridir. Ev adresi boş olduğundan kayıt doğrudan buraya yerleşmektedir ve hiçbir taşıma değerlendirilmemektedir. Bu adımdan sonra tabloda altı kayıt bulunmaktadır. Taşıma yalnızca ev adresi dolu olduğunda gündeme gelmektedir.

ADIM 7 / 9

## Brent's metodu: 16




Yedinci adımda 16 anahtarı eklenmektedir. Home address (ev adresi) on bire bölümden kalan olan 5, artım ise bölümün kendisi olan 1 değeridir. Ev adresi 27 anahtarıyla doludur. Yeni kaydın birincil zincirinde boş yer daha uzakta olduğundan, yerleşik kaydın kendi ikincil zinciri üzerinde taşınması değerlendirilmektedir. 27 anahtarının artımı 2 olduğundan 3 adım sonra 0 adresine ulaşmaktadır ve bu adres boştur. i ile j toplamı birincil zincirde yeni kaydın yerleşeceği probe sayısından küçük olduğu için taşıma kazançlıdır. 27 taşınmakta ve 16 ev adresine yazılmaktadır.

ADIM 8 / 9

## Brent's metodu: 42




Home address (ev adresi) 9 dolu, artım 3

Birincil zincirde ilk boş yer 1: taşıma kazandırmaz

65

Sekizinci adımda 42 anahtarı eklenmektedir. Home address (ev adresi) on bire bölümden kalan olan 9, artım ise bölümün kendisi olan 3 değeridir. Ev adresi doludur. Birincil zincir üzerinde artım 3 ile ilerlendiğinde ilk boş yer 1 adresinde, ikinci probe'ta bulunmaktadır. s değeri iki olduğundan i ile j toplamının ikiden küçük olması olanaksızdır ve taşıma denenmemektedir. Kayıt birincil zincirdeki boş yere yazılmaktadır.

ADIM 9 / 9

## Brent's metodu: 17




Dokuzuncu adımda 17 anahtarı eklenmektedir. Home address (ev adresi) on bire bölümden kalan olan 6, artım ise bölümün kendisi olan 1 değeridir. Ev adresi 39 anahtarıyla doludur. Yeni kaydın birincil zincirinde boş yer daha uzakta olduğundan, yerleşik kaydın kendi ikincil zinciri üzerinde taşınması değerlendirilmektedir. 39 anahtarının artımı 3 olduğundan 3 adım sonra 4 adresine ulaşmaktadır ve bu adres boştur. i ile j toplamı birincil zincirde yeni kaydın yerleşeceği probe sayısından küçük olduğu için taşıma kazançlıdır. 39 taşınmakta ve 17 ev adresine yazılmaktadır.

ÖZET

## Adım adım yerleştirme

$H_1(k) = k \bmod 11$  $H_2(k) = \text{bölüm}(k / 11)$

| Anahtar | Ev adresi | Artım | Yapılan işlem | Yerleşti |
|---|---|---|---|---|
| 27 | 5 | 2 | ev adresi boş | 5 |
| 18 | 7 | 1 | ev adresi boş | 7 |
| 29 | 7 | 2 | birincil zincir, s = 2 | 9 |
| 28 | 6 | 2 | ev adresi boş | 6 |
| 39 | 6 | 3 | 28 → 8 taşındı (i=1, j=1) | 6 |
| 13 | 2 | 1 | ev adresi boş | 2 |
| 16 | 5 | 1 | 27 → 0 taşındı (i=1, j=3) | 5 |
| 42 | 9 | 3 | birincil zincir, s = 2 | 1 |
| 17 | 6 | 1 | 39 → 4 taşındı (i=1, j=3) | 6 |



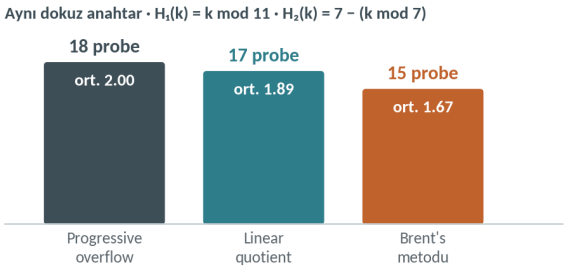
Örnekte $H_1$ anahtarın on bire bölümünden kalanı, $H_2$ ise bölümün kendisini vermektedir. İlk dört anahtar home address (ev adreslerine) ya da kısa zincirlerle yerleşmektedir. 39 eklenirken ev adresi 6 doludur. Yerleşik 28, kendi ikincil zinciri üzerinde bir adım ilerleyip 8'e taşınabildiğinden 39 ev adresine yerleşir. 16 eklenirken 27'nin üç adım taşınması gerekmekte, taşıma gene de birincil zincirdeki altıncı probe'tan ucuz kalmaktadır. Aynı durum 17 için de yinelenir.




Dokuz anahtarın tamamı yerleştiğinde toplam probe sayısı on sekiz, ortalama iki olmaktadır. Tabloda taşınan kayıtlar üstü çizili gösterilmiştir. Elde edilen düzen ÖA'nın çözümlü örneğiyle birebir aynıdır. Dikkat edilmesi gereken nokta, taşınan kaydın da kendi home address (ev adresinden) başlayan zincir üzerinde bulunabilir kalmasıdır. Arama algoritması değişmemekte, yalnızca yerleştirme algoritması değişmektedir. Adımlardaki taşımalar sayesinde uzun zincirlerin sonuna atılması gereken kayıtlar ev adreslerine yerleşmiş, taşınan kayıtların arama maliyeti ise yalnızca bir iki probe artmıştır.

KARŞILAŞTIRMA

## Üç yöntem, aynı anahtarlar




Anahtarlar: 71, 23, 73, 99, 44, 49, 60, 20, 33

Kazanç yerleştirmede yapılan fazladan işten geliyor

Eklemenin seyrek, aramanın sık olduğu dosyalarda ödünleşim Brent's lehine

69

Geçen haftanın anahtar kümesi üç yöntemle de işlendiğinde sıralama netleşmektedir. Progressive overflow on sekiz probe ile ortalama ikiyi, linear quotient on yedi probe ile bir virgül seksen dokuzu, Brent's metodu on beş probe ile bir virgül altmış yediyi vermektedir. Kazanç, yerleştirme sırasında yapılan fazladan işten gelmektedir. Arama maliyeti düşerken ekleme maliyeti yükselmektedir. Eklemenin seyrek, aramanın sık olduğu dosyalarda ödünleşim Brent's yönteminin lehinedir.

ALIŞTIRMA 3 · SORU

## Brent's metodu

$H_1(k) = k \bmod 10$ · $H_2(k) = \text{bölüm}(k / 10)$

Ev adresi doluysa önce birincil zincir izlenir, s probe sayısı bulunur

i + j < s olan bir taşıma varsa uygulanır; Probe sütunu tablonun son durumuna göre doldurulur

ANAHTARLAR · BU SIRAYLA EKLENİR

35 43 70 63 30 14 23 24 34

| Anahtar | Ev | Artım | İşlem | Yerleşti | Probe |
|---|---|---|---|---|---|
| 35 | ? | ? | ? | ? | ? |
| 43 | ? | ? | ? | ? | ? |
| 70 | ? | ? | ? | ? | ? |
| 63 | ? | ? | ? | ? | ? |
| 30 | ? | ? | ? | ? | ? |
| 14 | ? | ? | ? | ? | ? |
| 23 | ? | ? | ? | ? | ? |
| 24 | ? | ? | ? | ? | ? |
| 34 | ? | ? | ? | ? | ? |

70

Bu alıştırmada Brent's metodu on gözlü bir tabloda çalıştırılmaktadır. Birinci hash function (hash fonksiyonu) onluk bölmeden kalan, ikinci fonksiyon ise anahtarın ona bölümüdür ve artımı verir. Öğrencilerden her anahtar için ev adresini ve artımı yazmaları, collision (çarpışma) durumunda önce birincil zincirdeki probe sayısını bulmaları, ardından yerleşik kayıtlardan birinin taşınmasıyla daha kısa bir yol elde edilip edilemeyeceğini sınamaları beklenmektedir. Probe sütunu, taşınan kayıtların yeni yerleri hesaba katılarak tablonun son durumuna göre doldurulmalıdır. Tablo boyunun asal olmamasının etkisi çözümden sonra tartışılabilir.

ALIŞTIRMA 3 · ÇÖZÜM

## Brent's metodu

| Anahtar | Ev | Artım | İşlem | Yerleşti | Probe |
|---|---|---|---|---|---|
| 35 | 5 | 3 | ev adresi boş | 5 | 1 |
| 43 | 3 | 4 | ev adresi boş | 3 | 3 |
| 70 | 0 | 7 | ev adresi boş | 0 | 2 |
| 63 | 3 | 6 | birincil zincir, s = 2 | 9 | 2 |
| 30 | 0 | 3 | 70: 0 → 7 taşındı | 0 | 1 |
| 14 | 4 | 1 | ev adresi boş | 4 | 5 |
| 23 | 3 | 2 | 43: 3 → 1 taşındı | 3 | 1 |
| 24 | 4 | 2 | birincil zincir, s = 2 | 6 | 2 |
| 34 | 4 | 3 | 14: 4 → 8 taşındı | 4 | 1 |

Brent's toplam 18 probe · linear quotient 22 probe

Tablo boyu 10 asal değil: artım 2, 4, 6 bütün adresleri dolaşamaz



Dokuz anahtarın üçünde taşıma yapılmaktadır. 30 eklenirken sıfırıncı adresteki 70 yedinci adrese, 23 eklenirken üçüncü adresteki 43 birinci adrese, 34 eklenirken dördüncü adresteki 14 sekizinci adrese taşınmaktadır. Taşınan kayıtların arama maliyeti artmakta, buna karşılık yeni gelen anahtarlar ev adreslerine yerleşmektedir. Tablonun son durumunda toplam probe sayısı on sekizdir. Aynı anahtarlar linear quotient (doğrusal bölüm) ile yerleştirildiğinde toplam yirmi iki probe yapılmaktadır. Tablo boyu on olduğu için iki, dört ve altı gibi artımlar bütün adresleri dolaşamaz; bu nedenle gerçek uygulamada tablo boyu asal seçilmelidir.

6

# Uygulama

Kabuk oturumu ve algoritmaların C++ gerçeklemesi

UYGULAMA · 1

## File system (dosya sistemi) yapılarının gözlenmesi

```
$ ls -i dosya dosya2
67158084 dosya   67158084 dosya2

$ ln -s dosya dosya3
$ ls -li
67158084 -rw-r--r--  2 emir  ...  dosya
67158084 -rw-r--r--  2 emir  ...  dosya2
67158091 lrwxr-xr-x  1 emir  ...  dosya3 -> dosya

$ df -i .
$ du -h --max-depth=1
```

Hard link (sabit bağlantıda) inode numarası aynı, türü normal dosya

Symbolic link (sembolik bağlantıda) numara farklı, tür l ile gösterilir



Kabuk komutları, file system (dosya sisteminin) iç yapısını doğrudan gözlemlemeye elverir. Dosya bilgisi veren komut inode numarasını ve bağlantı sayısını gösterir. Bağlantı kurulduğunda iki adın aynı inode numarasını taşıdığı, sayacın arttığı ve adlar kaldırıldıkça gerilediği izlenir. Symbolic link (sembolik bağlantıda) ise inode numarası farklıdır ve dosya türü ayrı görünür. Aynı oturum, bölüm birde anlatılan kavramların sınıfta doğrulanmasını sağlar.

C++ · 1

# Coalesced hashing (birleştirilmiş hashing) · ekleme

```
struct Goz { int deger = BOS; int baglanti = -1; };

int ekle(vector<Goz>& t, int anahtar, bool erken) {
    const int n = static_cast<int>(t.size());
    const int ev = anahtar % n;
    if (t[ev].deger == BOS) { t[ev].deger = anahtar; return ev; }

    int bos = n - 1;
    while (bos >= 0 && t[bos].deger != BOS) --bos;
    if (bos < 0) return -1;
    t[bos].deger = anahtar;

    if (erken) {
        t[bos].baglanti = t[ev].baglanti;
        t[ev].baglanti  = bos;
    } else {
        int son = ev;
        while (t[son].baglanti != -1) son = t[son].baglanti;
        t[son].baglanti = bos;
    }
    return bos;
}
```

İki değişke arasındaki fark yalnızca bağlantıların kurulduğu satırlardadır

ÇÖZÜMLÜ ÖRNEK

# LISCH ve EISCH aynı anahtarlarla

```
int probe(const vector<Goz>& t, int k) {
    int a = k % int(t.size()), p = 1;
    while (t[a].deger != k) { a = t[a].baglanti; ++p; }
    return p;
}
vector<Goz> t(11);
for (int k : anahtarlar) ekle(t, k, erken);
for (int k : anahtarlar) toplam += probe(t, k);
for (int ev : {5, 6, 7}) {
    cout << "  " << ev << ":";
    for (int a = ev; a != -1; a = t[a].baglanti)
        cout << ' ' << t[a].deger;
}
```

ÇIKTI

```
LISCH: toplam 16, 17 icin 4
  5: 27 16
  6: 28 39 42 17
  7: 18 29
EISCH: toplam 15, 17 icin 2
  5: 27 16
  6: 28 17 39 42
  7: 18 29
```

Toplam 16 ve 15: elle izlemeyle aynı

C++ · 2

# Brent's metodu · ekleme

```
constexpr int BOS = -1;

void brent_ekle(vector<int>& t, int k, int (*h2)(int)) {
    const int m = t.size(), ev = k % m, adim = h2(k);
    if (t[ev] == BOS) { t[ev] = k; return; }
    int s = 2, yer = (ev + adim) % m;
    while (t[yer] != BOS) { yer = (yer + adim) % m; ++s; }
    for (int top = 2; top < s; ++top)
        for (int i = 1; i < top; ++i) {
            int j = top - i;
            int p = (ev + (i - 1) * adim) % m;
            int hedef = (p + j * h2(t[p])) % m;
            if (t[hedef] == BOS) {
                t[hedef] = t[p];  t[p] = k;  return;
            }
        }
    t[yer] = k;
}
```

Taşıma yalnızca i + j < s ise yapılır

ÇÖZÜMLÜ ÖRNEK

# Brent'ş metodu, dokuz anahtar

```
int h2(int k) { return k / 11; }
vector<int> t(11, BOS);
for (int k : {27, 18, 29, 28, 39, 13, 16, 42, 17}) brent_ekle(t, k, h2);
for (int a = 0; a < 11; ++a) {
    if (t[a] == BOS) continue;
    int k = t[a], p = k % 11, n = 1;
    while (p != a) { p = (p + h2(k)) % 11; ++n; }
    toplam += n;
    cout << a << ':' << k << '(' << n << ')' << (++say % 5 ? "  " : "\n");
}
```

ÇIKTI

```
0:27(4)  1:42(2)  2:13(1)  4:39(4)  5:16(1)
6:17(1)  7:18(1)  8:28(2)  9:29(2)
toplam 18, ortalama 2
```

Toplam 18, ortalama 2: ÖA'nın tablosu

Taşınan 27, 28 ve 39 hâlâ bulunabilir